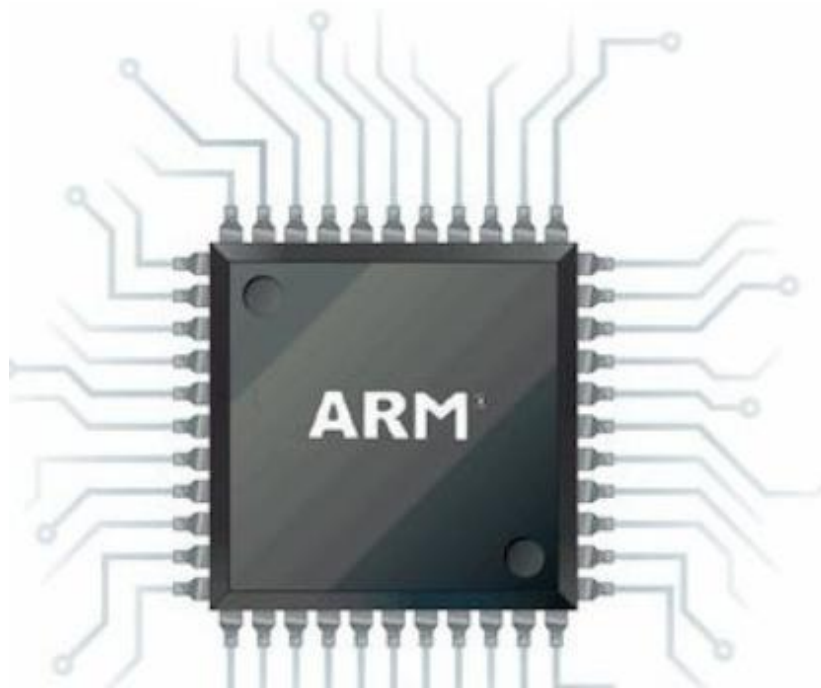




第2章 ARM处理器及系统结构



本章主要内容

2.1 ARM 处理器简介

2.2 ARM 处理器系列

2.3 ARM9 处理器内核

2.4 ARM 处理器工作状态

2.5 ARM 处理器运行模式

2.6 ARM 寄存器

2.7 ARM 存储系统

2.8 中断和异常的基本概念

2.9 ARM 的异常

本章教学要求

1、了解

- ARM处理器
- ARM处理器系列

2、理解

- ARM9 处理器内核
- ARM存储系统

3、掌握

- ARM处理器工作状态
- ARM处理器运行模式
- ARM寄存器
- 中断和异常的基本概念
- ARM的异常

2.1 ARM 处理器简介

ARM (Advanced RISC Machines) 有 3 种含义:

- 它是一个公司的名称;
- 还是一种技术的名称
- 是一类微处理器的通称;

基于 ARM技术 的微处理器应用约占据了 32位RISC 微处理器 80% 以上的市场份额, 其中, 在手机市场, ARM 占有绝对的垄断地位。

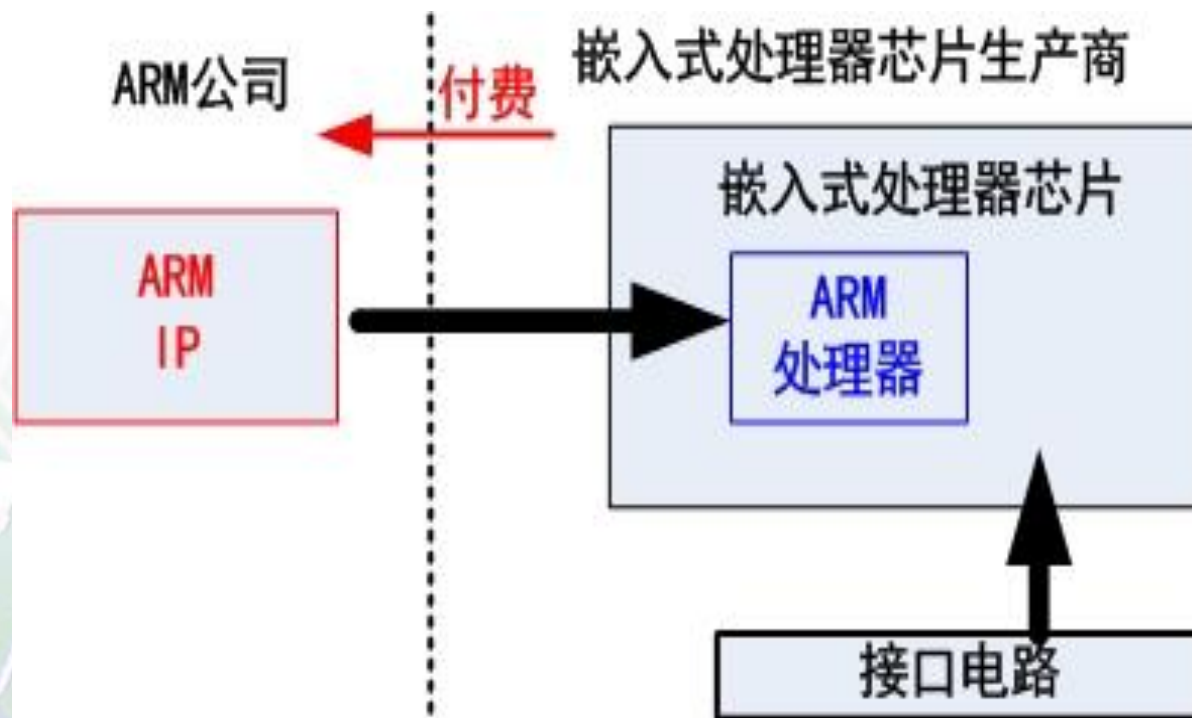
2.1 ARM 处理器简介

ARM公司 是专门从事基于 RISC 技术芯片开发的公司，作为嵌入式 RISC 处理器的知识产权 IP 供应商。

使用 ARM 公司硬件技术授权的公司有：Intel、IBM、SAMSUNG、LG半导体、NEC、SONY、PHILIP、华为等几十多家企业和公司。至于软件系统的合伙人，则包括Microsoft、SUN等一系列知名公司。其合作伙伴超过超过200家。

2.1 ARM 处理器简介

ARM公司 与 芯片厂商 的关系



2.1 ARM 处理器简介

采用 RISC 架构的 ARM 微处理器一般有如下特点：

- ① 体积小、功耗低、低成本、高性能；
- ② 支持 Thumb（16位）/ARM（32位）双指令集，能很好地兼容 8位/16位 器件；
- ③ 大量使用 寄存器，指令执行速度更快；
- ④ 大多数数据操作都在寄存器中完成；
- ⑤ 寻址方式灵活简单，执行效率高；
- ⑥ 指令长度固定。

2.1 ARM 处理器简介

到目前为止，ARM 微处理器及技术的应用几乎已经深入到各个领域：

- (1) 工业控制领域；
- (2) 无线通讯领域；
- (3) 消费类电子产品；
- (4) 成像和安全产品。

2.2 ARM 微处理器系列

目前常用基于ARM核的处理器有以下几类：ARM7系列、ARM9系列、ARM9E系列、ARM10E系列、ARM11系列、SecurCore微处理器系列、Coretex等。

2.2 ARM 微处理器系列

2.2.1 ARM指令集体系结构的演变（了解）

ARM公司定义了不同版本的ARM指令集体系结构版本，用vn来标识（n是版本号，目前是1-8）。

ARM v1 ~ v3版本的处理器未得到大量应用，ARM处理器的大量广泛应用是从其v4版本开始的。

v7系列CPU被称作CoreTex系列

到目前v8是最新的版本， 可以选择64或32执行状态。64执行状态针对64位处理技术，引入了一个全新指令集A64，可以存取大虚拟地址空间。

2.2 ARM 微处理器系列

一、ARMv4

ARMv4是目前支持的最老的架构,是基于32-bit地址空间的32-bit指令集。ARMv4除了支持ARMv3的指令外还扩展了:

- 支持halfword的存取
- 支持byte和halfword的符号扩展读
- 进一步的明确了会引起Undefined异常的指令
- 对以前的26bits体系结构的CPU不再兼容

2.2 ARM 微处理器系列

二、ARMv4T

ARMv4T增加了16-bit Thumb 指令集，这样使得编译器能产生紧凑代码（相对于32-bit代码，内存能节省到35%以上）并保持32-bit系统的好处。

Thumb在处理器中仍然要扩展为标准的32位ARM指令来运行。用户采用16位Thumb指令集最大的好处就是可以获得更高的代码密度和降低功耗。

2.2 ARM 微处理器系列

三、ARMv5TE

1999年推出ARMv5TE其增强了Thumb体系,增强的Thumb体系增加了新的指令,同时改进了Thumb/ARM**相互作用、编译能力和混合及匹配**ARM与Thumb例程,以更好地平衡**代码空间和性能**,并在ARM指令集体系结构上扩展了增强的DSP 指令集:

增强的DSP指令包括支持饱和算术(saturated arithmetic),并且针对Audio DSP应用提高了70%性能。

'E' 扩展表示在通用的CPU上提供DSP能力。

2.2 ARM 微处理器系列

四、ARMv5TEJ

2000年推出ARMv5TEJ，增加了Jazelle扩展以支持Java加速技术（处理器指令层对JAVA加速）。

Jazelle技术比仅仅基于软件的JVM性能提高近8倍的性能减少了80%的功耗。

2.2 ARM 微处理器系列

五、ARMv6

2001年推出ARMv6，它在许多方面做了改进如内存系统、异常处理和较好地支持多处理器。

SIMD扩展使得广大的软件应用如Video和Audio codec的性能提高了4倍。

Thumb-2和TrustZone 技术也用于ARMv6中。

ARMv6第一个实现是2002年春推出的ARM1136J(F)-STM 处理器，2003年又推出了 ARM1156T2(F)-S 和 ARM1176JZ(F)-S处理器。

2.2 ARM 微处理器系列

六、ARMv7

ARMv7 定义了3种不同的处理器配置（processor profiles）：

- Profile A是面向复杂、基于虚拟内存的OS和应用的
- Profile R是针对实时系统的
- Profile M是针对低成本应用的优化的微控制器的。

所有ARMv7 profiles实现Thumb-2技术。同时还包括了NEON™技术的扩展提高DSP和多媒体处理吞吐量400%，并提供浮点支持以满足下一代3D图形和游戏以及传统嵌入式控制应用的需要。

2.2 ARM 微处理器系列

七、ARMv8

2011年11月，ARM公司发布了新一代处理器架构ARM V8，**ARM的首个64位架构**。

在继承了v7架构的基础上，可以选择64或32执行状态。64执行状态针对64位处理技术，引入了一个全新指令集A64，可以存取大虚拟地址空间；

运用ARMv8指令集的产品：

ARM 2012年十月发布了Cortex-A53 and Cortex-A57 核心。2013年苹果基于V8架构的Apple 7使用在了iPhone 5S上。

2.2 ARM 微处理器系列

八、ARM处理器和指令集构架版本的对应关系

ARM处理器和指令集构架版本的对应关系具体的对应关系如下表：

2.2 ARM 微处理器系列

版本	版本变种	系列号	处理器核
V1	V1	ARM1	ARM1
V2	V2	ARM2	ARM2
	ARM2aS		
	V2a	ARM3	ARM3
V3	V3	ARM6	ARM6、ARM600、ARM610
		ARM7	ARM7、ARM700、ARM710
V4T			ARM7TDMI、ARM710T、ARM720T、ARM740T
V4	V4	ARM8	StrongARM、ARM8、ARM810
	V4T	ARM9	ARM9TDMI、ARM920T、ARM940T
V5	V5TE		
		ARM10	ARM10TDMI、ARM1020E
V6	V6	ARM11	ARM11、ARM11562-S、ARM1156T2F-S、ARM11JZF-S
V7	V7	ARM Cortex	ARM Cortex-A8、ARM Cortex-R4、ARM Cortex-M3

2.2 ARM 微处理器系列

Previous	Armv6	Armv7	Armv8		
Armv5	Armv6	Armv7-A	Armv8-A		
Arm968E-S Arm946E-S Arm926EJ-S	Arm11MPCore Arm1176JZ(F)-S Arm1136J(F)-S	Cortex-A17 Cortex-A15	Cortex-A73 Cortex-A57	Cortex-A75 Cortex-A72	High performance
		Cortex-A9 Cortex-A8	Cortex-A53	Cortex-A55	High efficiency
		Cortex-A7 Cortex-A5	Cortex-A35 Cortex-A32		Ultra high efficiency
Armv4	Armv6-M	Armv7-R	Armv8-R		
		Cortex-R8 Cortex-R7 Cortex-R5 Cortex-R4	Cortex-R52		Real time
		Armv7-M	Armv8-M		
Arm7TDMI Arm920T	Cortex-M	Cortex-M7 Cortex-M4 Cortex-M3	Cortex-M33		High performance Performance efficiency
		Cortex-M0+ Cortex-M0	Cortex-M23		Lowest power and area

2.2 ARM 微处理器系列

九、ARM/Thumb的命名规范

由于ARM的每个版本都有一系列的可变版本，其命名规范（Classic命名）如下：

1. 以ARMv开头
2. ARM指令集的版本号（1 - 8）
3. 可变功能标识（T—表示支持Thumb指令，M---表示支持乘法指令，E — 增强型DSP指令，J — Java加速器Jazelle, SIMD — ARM媒体功能扩展），但是ARMv4和其上的版本缺省都支持M功能，因此ARMv4和其上的版本的标识中可以不用标识M。
4. 如果3中描述的功能不存在，则在该功能标识符前加x。

例如，ARMv5TxM 表示 ARM 指令集版本为 5，支持 T 变种，不支持 M 变种。

2.2 ARM 微处理器系列

十、ARM处理器命名

规则：ARM{x}{y}{z}{T}{D}{M}{I}{E}{J}{F}{-S}

x -- 处理器系列

y -- 存储管理/保护单元

z -- cache

T -- 支持Thumb指令集

D -- 支持片上调试

M -- 支持快速乘法器

I -- 支持Embedded ICE, 支持嵌入式跟踪调试

E -- 支持增强型DSP指令

J -- 支持Jazelle

F -- 具备向量浮点单元VFP

-S -- 可综合版本

2.2 ARM 微处理器系列

ARM 926EJ-S

Family number

7: ARM7

9: ARM9

10: ARM10

11: ARM11

Memory system

2: Cache, MMU, Process ID

4: Cache, MPU

6: Write buffer, no cache

Memory size

0: Cache size (4-128KB)

2: Reduced cache size

6: TCM (紧耦合内存)

Synthesizable

Extensions

E: DSP extension

J: Jazelle extension

T: Thumb support

...

2.2 ARM 微处理器系列

十一、ARM处理器指令集的特点

ARM内核不是一个纯粹的RISC体系结构，ARM指令集与纯粹的RISC的定义有以下几个不同。

- ① 一些特定指令的周期数可变，并不是每条ARM指令都是单周期的。
- ② 内嵌的桶形移位器产生了更为复杂的指令，扩展了指令的功能，因此改善了内核的性能。
- ③ 支持16位的Thumb指令集，提高了代码密度。
- ④ 支持条件执行：每条指令都可以设置一个执行条件，只有条件满足时才执行。
- ⑤ 增强指令：一些功能强大的数字信号处理指令被加入到ARM指令集中。

2.2 ARM 微处理器系列

2.2.2 ARM体系结构的演变

(1) ARM7系列

ARM7系列微处理器为低功耗32位RISC处理器，最适合用于对价格和功耗要求较高的应用领域。

ARM7 系列包括：ARM7TDMI、ARM7TDMI-S、带有高速缓存处理宏单元的ARM720T 和 扩充了Jazelle的ARM7EJ-S，如下表所示：

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory Mgt	BUS 接 口	Thumb	DSP	Jazelle
ARM7TDMI	—	—	—	yes	yes	no	no
ARM7TDMI-S	—	—	—	yes	yes	no	no
ARM720	8KB		MMU	AHB	yes	no	no
ARM7EJ-S	—	—	—	yes	yes	yes	yes

2.2 ARM 微处理器系列

ARM7 微处理器系列具有如下特点：

- ① 具有嵌入式ICE JTAG 软件调试方式，调试开发方便；
- ② 极低的功耗，适合对功耗要求较高的应用，如便携式产品；
- ③ 能够提供0.9MIPS/MHZ的三级流水线结构（取指、译码、执行）；
- ④ 代码密度高并兼容16位的Thumb指令集；
- ⑤ 指令系统与ARM9、ARM9E和ARM10E系列兼容，便于用户产品的升级换代；
- ⑥ 主频最高可达130MIPS，高速的运算处理能力能胜任绝大多数的复杂应用。

2.2 ARM 微处理器系列

(2) ARM9系列

1997年问世，采用5级指令流水线。存储器系统根据哈佛体系结构（程序和数据空间独立的体系结构）重新设计，区分了数据总线和指令总线。

ARM9系列的第一个处理器是ARM920T，它包含独立的数据指令Cache和MMU。

该处理器能够用在要求有虚拟存储器支持的操作系统上。该系列中的 ARM922T是ARM920T的变种，只有一半大小的数据指令Cache。

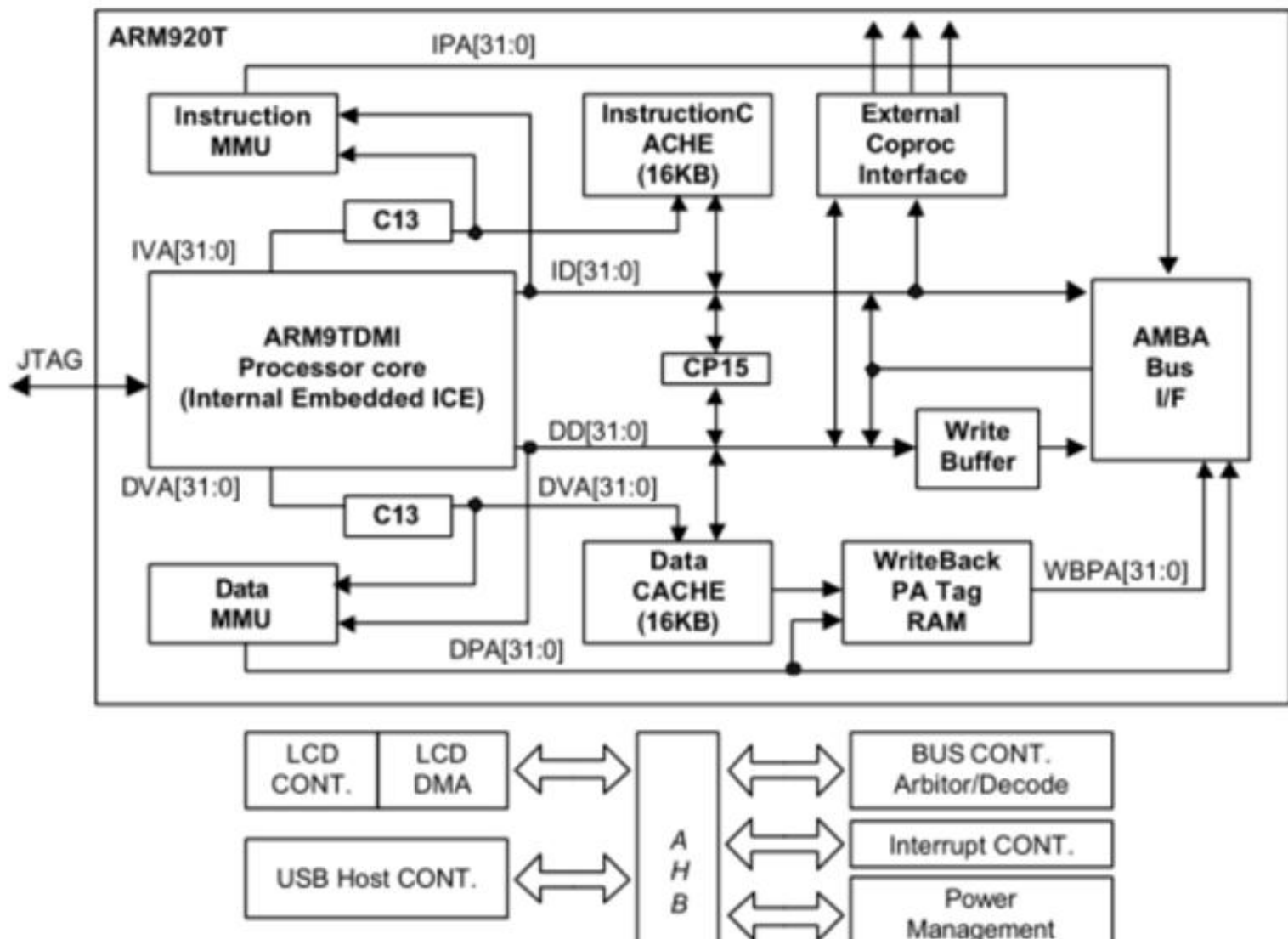
2.2 ARM 微处理器系列

ARM940T 包含一个更小的 数据指令 Cache 和一个 MPU(Micro Processor Unit)。它是针对不要求运行操作系统的应用而设计的。ARM920T、ARM940T都执行 **v4T** 架构指令。

ARM9系列产品

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory <u>Mgt</u>	BUS 接 口	Thumb	DSP	<u>Jazelle</u>
ARM920T	16KB/16KB	—	MMU	ASB	yes	no	no
ARM9TDMI	—	—	—	ASB	yes	no	no
ARM922T	8KB/8KB	—	MMU	ASB	yes	no	no
ARM940T	Fixed	—	MPU	ASB	yes	no	no

2.2 ARM 微处理器系列



2.2 ARM 微处理器系列

ARM9 系列微处理器具有以下特点：

- ① 5级整数流水线（取指、译码、执行、缓冲/数据、回写），指令执行效率高；
- ② 基于嵌入式ICE JTAG 的软件调试方式，调试开发方便；
- ③ 提供1.1MIPS/MHZ的哈佛结构；
- ④ 支持32位ARM指令集和16位Thumb指令集；
- ⑤ 支持32位的高速AMBA总线接口；
- ⑥ 全性能的MMU，支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统；

2.2 ARM 微处理器系列

- ⑦ MPU支持实时操作系统;
- ⑧ 支持 数据Cache 和 指令Cache, 具有更高的指令和数据处理器能力。

ARM9 系列兼容了 ARM7 系列, 主要用于: 仪器仪表、安全系统、引擎管理、机顶盒、高端打印机、PDA、网络电脑以及带有MP3音频和MPEG4视频多媒体格式的智能电话中。

2.2 ARM 微处理器系列

(3) ARM9E 系列

ARM9E 系列微处理器为综合处理器，使用单一的处理器内核，提供了微控制器、DSP、java应用系统解决方案，极大地减少了芯片的面积和系统的复杂程度。ARM9E系列包括ARM926EJ-S、ARM946E-S、ARM966E-S和ARM968E-S，其产品如下：

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory <u>Mgt</u>	BUS 接 口	Thumb	DSP	<u>Jazelle</u>
ARM926EJ-S	可变	yes	MMU	2XAHB	yes	yes	yes
ARM946E-S	可变	yes	MPU	AHB	yes	yes	no
ARM966E-S	—	yes	—	AHB	yes	yes	no
ARM968E-S 32	—	yes	DMA	AHB- Lite	yes	yes	no

2.2 ARM 微处理器系列

ARM9E 系列微处理器的主要特点如下：

- 支持**DSP指令集**，适合于需要高速数字信号处理的场合；
- 具有嵌入式ICE-RT逻辑，更好地适应实时系统开发；
- 5级整数流水线，指令执行效率高；
- 支持32位ARM指令集和16位Thumb指令集；
- 支持32位的高速AMBA总线接口；
- 支持**VFP9浮点处理协处理器**；
- 全性能的MMU，支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统；

2.2 ARM 微处理器系列

- MPU支持实时操作系统;
- 支持数据Cache和指令Cache, 具有更高的指令和数据处理能力;
- 主频最高可达**300MIPS**。

2.2 ARM 微处理器系列

ARM9E 系列处理器主要应用于下面一些场合：

- 下一代无线设备，包括视频电话和PDA；
- 数字消费品，包括：机顶盒、家庭网关、MP3播放器和MPEG-4播放器；
- 成像设备，包括：打印机、数码照相机和数码摄像机；
- 存储设备，包括 DVD 或 HDD 等；
- 工业控制，包括电机控制等；
- 汽车、通信和信息系统的 ABS 和 车体控制；
- 网络设备，包括：VoIP、WirelessLAN等。

2.2 ARM 微处理器系列

(4) ARM10E 系列

ARM10E 系列微处理器由于采用了新的体系结构，与同等的 ARM9 器件相比较，在同样的时钟频率下，性能提高了近 50%，同时还采用了两种先进的节能方式，使其功耗极低。ARM10E 系列包括 ARM1020E、ARM1022E 和 ARM1026EJ-S 三种类型，如下表所示：

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory Mgt	BUS 接口	Thumb	DSP	<u>Jazelle</u>
ARM1020E	32KB/32KB		MMU	2XAHB	yes	yes	no
ARM1022E	可变 16KB/16KB	—	MPU	2XAHB	yes	yes	no
ARM1026EJ-S	可变	yes	MMU或MPU	2XAHB	yes	yes	yes

2.2 ARM 微处理器系列

ARM10E 系列微处理器的主要特点如下：

- 支持DSP指令集，适合于需要高速数字信号处理的场合；
- 6级整数流水线，指令执行效率更高；
- 支持32位的ARM指令和16位的Thumb指令；
- 支持32位的高速AMBA总线接口；
- 支持VFP10浮点处理协处理器；
- 全性能的MMU，支持Windows CE、Linux、Palm OS等多种主流嵌入式操作系统；

2.2 ARM 微处理器系列

- 支持数据Cache和指令Cache，具有更高的指令和数据处理能力；
- 主频最高可达400MIPS；
- 内嵌并行读/写操作部件。

2.2 ARM 微处理器系列

(5) ARM11系列

ARM11 系列的运行频率高达500~700MHZ，更为强大的性能。ARM11 处理器在增加流水线数的同时，还在结构设计方面进行了改进。

ARM11 包含64个4种状态的跳转目标地址缓存，用来存储最近使用过的跳转目标地址，这将会极大地提高IPC指标，尤其是对那些存在许多条件跳转指令的测试向量或应用程序。

ARM11 系列包括ARM1136(F)-S、ARM1156T2(F)-S和ARM1176JZ(F)-S三种类型，如下表所示：

2.2 ARM 微处理器系列

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory <u>Mgt</u>	BUS 接 口	Thumb	DSP	<u>Jazelle</u>
ARM1136J(F)-S	可变	yes	MMU	5XAHB	yes	yes	yes
ARM1156T2(F)-S	可变	yes	MPU	4XAXI	yes	yes	no
ARM1176JZ(F)-S	可变	yes	MMU	4XAXI	yes	yes	yes

ARM11 系列微处理器的主要特点如下：

- 工作频率高达700MHZ，功耗低至200mW；
- 实时响应快，中断延迟更低，支持矢量化中断，中断速度提高 3 倍；

2.2 ARM 微处理器系列

- 内核有8级流水线，带2个周期的高速闪存访问，可实现高频工作；
- ARM11处理器在提供高性能的同时，也允许在性能和功耗间做权衡以满足某些特殊应用。通过动态调整时钟频率和供应电压，开发者完全可以控制这两者的平衡。在0.13um工艺，1.2v条件下，ARM11处理器的功耗可以低至0.4mW/MHz.；
- ARM11 MPCore使用多核处理器结构，可实现从1个内核到4个内核的多核可扩展性，从而使得单个宏的简单系统设计可以具备4倍于单个内核的性能；

2.2 ARM 微处理器系列

- 存储器管理单元支持Microsoft Windows、Symbian OS、WinRiver和Linux操作系统;
- **多媒体处理扩展**: 使MPEG4编码/解码**加快一倍**、音频处理加快一倍;

2.2 ARM 微处理器系列

(6) Cortex 系列

ARM 推出的 Cortex 系列包括：Cortex-A、Cortex-R 和 Cortex-M三个系列。Cortex-A 系列主要应用于复杂的应用中，支持：ARM、Thumb 和 Thumb2指令集。

Cortex-R系列 是为实时操作系统设计的嵌入式处理器，能带来更小的芯片面积和低功耗。

Cortex-M系列 处理器结合了多种突破性技术，集成了许多紧耦合系统外设，以便能满足下一代产品的控制要求，适用于高性能、低成本需求的嵌入式应用。Cortex-M 主要针对 单片机领域。

2.2 ARM 微处理器系列

(7) SecurCore 微处理器系列

SecurCore 系列微处理器专为安全需要而设计，提供了完善的 32位 RISC 技术的安全解决方案。

SecurCore 系列微处理器包含 SecurCore SC100 , SecurCore SC110 , SecurCore SC200 和 SecurCore SC210 四种类型，如下所示：

型号	Cache (Inst/Data)	Tightly Coupled Memory	Memory Mgt	BUS 接口	Thumb	DSP	<u>Jazelle</u>
<u>SecurCore</u> SC100	—	—	MPU	—	yes	no	no
<u>SecurCore</u> SC110	—	—	MPU	—	yes	no	no
<u>SecurCore</u> SC200	—	—	MPU	—	yes	no	yes
<u>SecurCore</u> SC210	—	—	MPU	—	yes	no	yes

2.2 ARM 微处理器系列

SecurCore 系列处理器除了具有 ARM 体系结构的主要特点外，还在系统安全方面具有如下的特点：

- ① 带有灵活的保护单元，以确保操作系统和应用数据的安全；
- ② 采用软内核技术，防止外部对其进行扫描探测；
- ③ 可集成用户的安全特性和其他协处理器。

SecurCore 系列微处理器主要应用于一些对安全性要求较高的应用产品及应用系统，如：电子商务、电子政务、电子银行业务、网络和认证系统等领域。

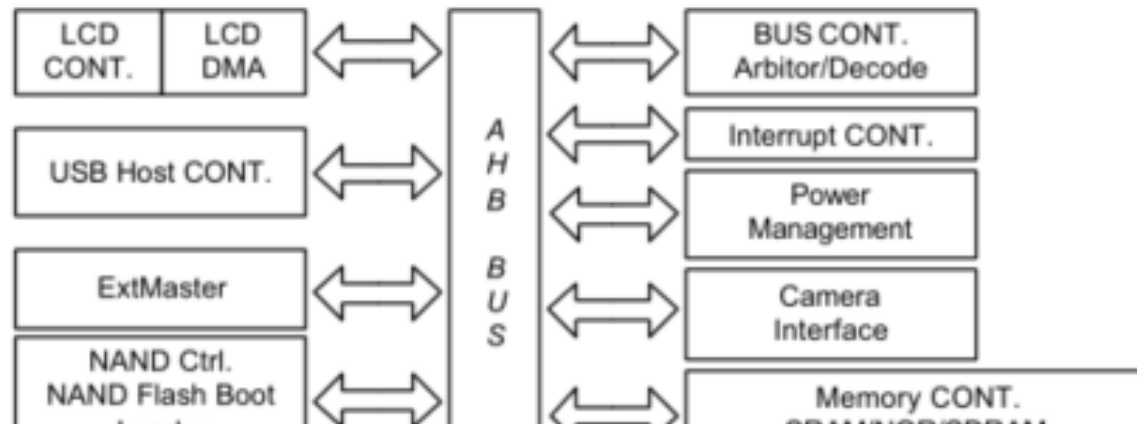
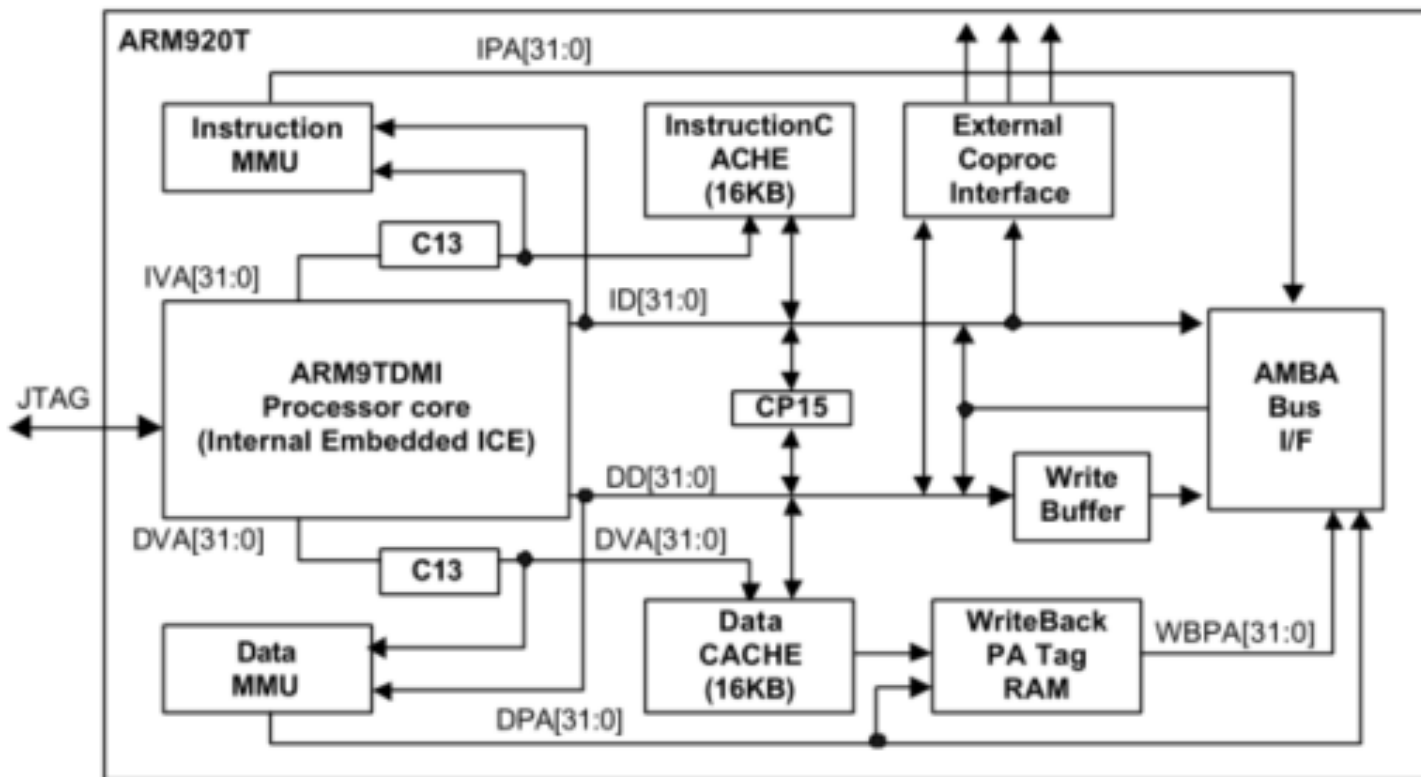
2.3 ARM9 处理器内核

ARM9系列有ARM920T、ARM922T和 带有高速缓冲器宏单元的ARM940T。全部的ARM9系列处理器均具有16位Thumb指令集和基于Embedded ICE的调试方式。

ARM920T、ARM922T和ARM940T是在ARM9TDMI的基础上研制开发的。

ARM9TDMI是基于ARM体系结构v4版本的高端ARM核（注意：核并非芯片，ARM核与其他部件（如RAM、ROM、片内外设）组合在一起才构成现实的芯片）。

2.3 ARM9 处理器内核



2.3 ARM9 处理器内核

2.3.1 ARM9系列MCU

一、ARM9TDMI

ARM9TDMI的后缀意义如下：

T：支持16位Thumb指令集；

D：支持片上Debug；

M：内嵌硬件乘法器；

I：嵌入式ICE，支持片上断点和调试点。

- ARM9TDMI使用哈佛结构，通过cache可以同时读取指令和数据。
- 流水线从ARM7TDMI的 3级 增加到 5级。
- 在相同工艺下，ARM9TDMI的性能近似为ARM7TDMI的2倍。

2.3 ARM9 处理器内核

二、ARM920T

ARM920T核由ARM9TDMI、存储管理单元MMU和高速缓存三部分组成。其中，MMU可以管理虚拟内存，高速缓存由独立的16KB指令和16KB数据高速Cache组成。

ARM920T有两个内部协处理器CP14和CP15。CP14用于调试控制，CP15用于存储系统控制及测试。

三、ARM940T

ARM940T内置指令和数据Cache、保护单元和高速AMBA总线接口。

2.3 ARM9 处理器内核

2.3.2 ARM9TDMI的流水线

一、ARM7的 3 级流水线



2.3 ARM9 处理器内核

二、ARM9的 5 级流水线

5级流水线 是 ARM9TDMI 的主要特征之一，其设计减少了在每个时钟内必须完成的最大工作量，进而允许使用较高的时钟频率。

- 取指：指令从存储器中取出，放入指令流水线；
- 译码：指令译码，从寄存器堆中读取寄存器操作数；
- 执行：把一个操作数移位，产生ALU的结果。如果指令是Load或Store，在ALU中计算存储器的地址；

2.3 ARM9 处理器内核

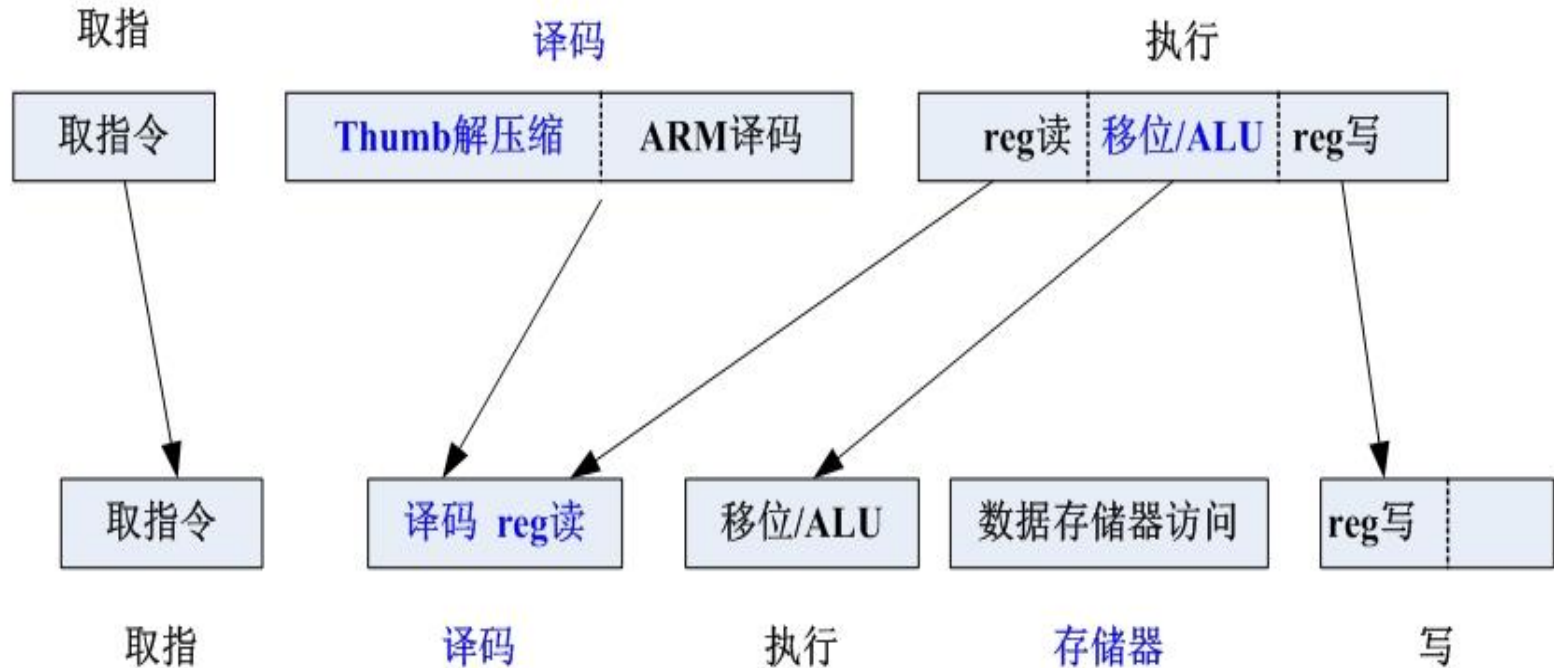
二、ARM9的 5 级流水线

- 缓存/数据：如果需要，则访问数据存储器；否则，ALU的结果只是简单地缓冲一个时钟周期，以便使所有指令具有同样的流水线流程；
- 回写：将指令产生的结果写回到寄存器堆，包括任何从寄存器读出的数据。

2.3 ARM9 处理器内核

下图为ARM9TDMI与ARM7TDMI流水线比较

ARM7TDMI



2.3 ARM9 处理器内核

指令流水的扩展讲解 (了解)

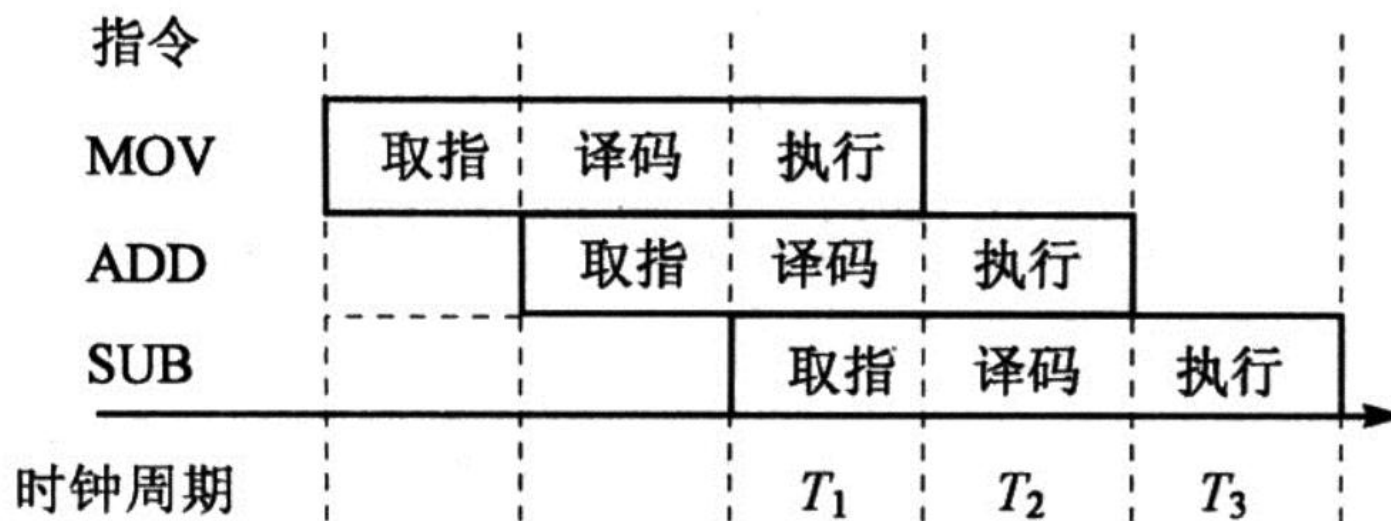


图 1 ARM7 单周期指令最佳流水线

2.3 ARM9 处理器内核

指令流水的扩展讲解 (了解)



图 2 带有存储器访问指令的流水线

2.3 ARM9 处理器内核

指令流水的扩展讲解 (了解)



图 5 ARM9 的五级最佳流水线

2.3 ARM9 处理器内核

2.3.3 协处理器接口

协处理器用于减轻系统微处理器的特定处理任务的芯片或模块。例如：intel pentium 微处理器就包括内置的数学协处理器。

ARM处理器内置的协处理器通过**扩展指令集**或提供**配置寄存器**来扩展内核处理功能。

扩展指令集

协处理器也能通过提供一组专门的新指令来扩展指令集。例如，有一组专门的指令可以添加到标准ARM指令集中，以处理向量浮点（VFP）运算。

2.3 ARM9 处理器内核

- 这些新指令是在ARM流水线的译码阶段被处理的。协处理器和ARM处理器连接到同一个指令总线，这意味着协处理器可以对指令流中的指令进行译码并执行其所支持的指令。
- 如果在译码阶段发现是一条协处理器指令，则把它交给相应的协处理器。
- 如果该协处理器不存在，或不认识这条指令，则ARM认为发生了未定义指令异常，则执行未定义指令陷阱（即未定义指令异常服务子程序）。

2.3 ARM9 处理器内核

可用的协处理器

一个系统中最多可连接16个协处理器。每个协处理器都由唯一的ID标识。

AMR920T处理器包含两个内部协处理器：

- (1) CP14通信通道协处理器，负责调试；
- (2) CP15为CACHE和MMU功能提供的系统控制协处理器。

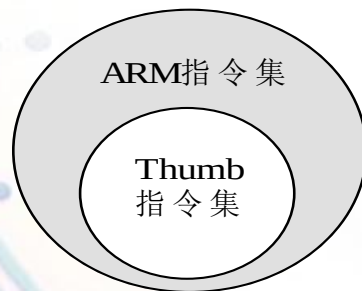
可用的协处理器

协处理器编号	分配	协处理器编号	分配
15	系统控制	7:4	可供芯片设计者使用
14	调试控制器	3:0	保留
13:8	保留		

2.4 ARM处理器工作状态

嵌入式系统在某些应用场合对存储成本或空间要求比较苛刻，为了让用户更好地控制代码量，于是设计了2套指令系统，分别为ARM指令集和Thumb指令集。其中ARM指令集为32位（字）长度，具有最完整的功能；Thumb指令集为16位（半字）长度，能实现ARM指令集的大部分功能。

在功能上可以认为Thumb是ARM指令集的子集（见下图）：



2.4 ARM处理器工作状态

既然ARM处理器共存2种指令集，那么到底何时执行ARM指令集，何时执行Thumb指令集呢？

ARM处理器有2个处理器状态与这2套指令集分别对应。

ARM9TDMI核是具有T变种的ARM处理器核，其工作状态一般有以下两种，并可在两种状态之间切换。

(1) ARM状态：

执行32位的字对齐的ARM指令。是系统上电默认的状态。

(2) Thumb状态：

执行16位的、半字对齐的Thumb指令。

2.4 ARM处理器工作状态

ARM处理器的两种状态可以通过相应的指令进行切换。

ARM指令集与Thumb指令集不能同时混合使用。

注意：ARM和Thumb状态间的切换并不影响处理器模式或寄存器内容。

从一个ARM例程调用另一个Thumb例程时，内核必须切换状态，反之亦然。下面使用BX分支指令将ARM内核的操作状态在ARM和Thumb之间进行切换，见程序清单。

2.4 ARM处理器工作状态

； 从 ARM状态 切换到 Thumb状态

CODE32

LDR R0, =Lable+1

BX R0

CODE16

Lable MOV R1,#12

； 下面的指令为 ARM指令

； R0的 bit0=1, BX自动将 CPSR中的 T置 1

； 切换到 Thumb状态, 并跳转到 Lable处 执行

； 下面的指令为 Thumb指令

； 从 Thumb状态 切换到 ARM状态

CODE16

LDR R0, =Lable

BX R0

CODE32

Lable MOV R1,#10

； 下面的指令为 Thumb指令

； R0的 bit0=0, BX自动将 CPSR中的 T置 0

； 切换到 ARM状态, 并跳转到 Lable处 执行

； 下面的指令为 ARM指令

2.4 ARM处理器工作状态

(3) 指令长度和数据格式

ARM微处理器的指令长度可以是32位（在ARM状态下），也可以为16位（在Thumb状态下）。

ARM微处理器中支持字节（8位）、半字（16位）、字（32位）三种数据类型。

- 如果一个数据是以字方式存储的，那么它就是字对齐的；
- 如果一个数据是以半字方式存储的，那么它就是半字对齐的。

方 式	半字对齐	字对齐
地 址	0x4002 0x4004 0x4006 0x4008 .	0x4004 0x4008 0x400C 0x4010 .
特 征	bit0=0 其他位置为任意值	bit1=0,bit0=0 其他位置为任意值

2.5 ARM处理器运行模式

ARM体系结构支持 **7种** 运行模式：

(1) 用户模式 (USR)：

正常程序执行模式，大部分任务执行在这种模式下。用户模式是用户程序的工作模式。该没有权限去操作其它硬件资源，也不能切换到其它模式下。

(2) 系统模式 (SYS)：

系统模式是特权模式，不受用户模式的限制。**用户模式和系统模式共用一套寄存器**，但比用户模式有更高的权限，可以访问所有系统资源及进行模式切换。

2.5 ARM处理器运行模式

ARM体系结构支持 7种 运行模式：

(3) 一般中断模式 (IRQ)：

一般中断模式也叫通用中断模式，用于处理一般的中断请求，通常在硬件产生中断信号之后自动进入该模式，该模式为特权模式，可以自由访问系统硬件资源。

(4) 快速中断模式 (FIQ)：

快速中断模式是相对一般中断模式而言的，它是用来处理对时间要求比较紧急的中断请求，主要用于高速数据传输及通道处理中，其中断优先级相对普通中断更高。

2.5 ARM处理器运行模式

ARM体系结构支持 **7种** 运行模式：

(5) 管理模式 (Supervisor, SVC) :

管理模式是CPU上电后默认模式，因此在该模式下主要用来做系统的初始化，软中断处理也在该模式下。**系统复位或开机、软中断时进入到SVC模式下。**

(6) 中止模式 (ABT) :

中止模式用于支持虚拟内存或存储器保护，当用户程序访问非法地址，没有权限读取的内存地址时，会进入该模式。

2.5 ARM处理器运行模式

ARM体系结构支持 **7种** 运行模式：

(7) 未定义模式 (UND)：

CPU在指令的译码阶段不能识别该指令操作时，会进入未定义模式，进行未定义陷阱处理。

特权模式

除用户模式外的其它6种处理器模式称为特权模式 (Privileged Modes)。在特权模式下，程序可以访问所有的系统资源，也可以任意的进行处理器模式切换。

只有在特权模式下才允许对当前程序状态寄存器 (CPSR) 的所有控制位直接进行读/写访问，而在非特权模式下只允许对CPSR的控制位进行间接访问 (SWI方式)。

2.5 ARM处理器运行模式

异常模式

特权模式中除系统模式之外的其他5种模式又统称为异常模式。它们除了可以通过在特权下的程序切换进入外，也可以由特定的异常进入。其中管理模式也称为超级用户模式，是为操作系统提供软中断的特有模式。

大多数的用户程序运行在用户模式下。当处理器工作在用户模式时，需要进行处理器模式切换时，应用程序可以产生异常处理，在异常处理过程中进行处理器模式切换。这种体系结构可以使操作系统控制整个系统资源的使用。

2.5 ARM处理器运行模式

处理器模式			说明	备注
用户（usr）			正常程序运行的工作模式	不能直接从用户模式切换到其他模式
特权模式	系统（sys）		用于支持操作系统的特权任务等	与用户模式类似，但具有直接切换到其他模式等特权
	异常模式	管理（svc）	供操作系统使用的一种保护模式	只有在系统复位和软件中断响应时，才进入此模式
		中止（abt）	用于虚拟内存和（或）存储器保护	对程序存储器或数据存储器访问失败时，进入此模式
		未定义（und）	支持软件仿真的硬件协处理器	只有在未定义指令异常响应时，才进入此模式
		中断（irq）	中断请求处理	只有在IRQ异常响应时，才进入此模式
		快速中断（irq）	快速中断请求处理	只有在FIQ异常响应时，才进入此模式

2.6 ARM寄存器

ARM处理器有如下37个用户可见寄存器。

(1) 31 个通用寄存器：R0~R15、R13_svc、R14_svc、R13_abt、R14_abt、R13_und、R14_und、R13_irq、R14_irq、R8_fiq~R14_fiq。

(2) 6 个状态寄存器：1 个CPSR、5 个SPSR (SPSR_svr、SPSR_abt、SPSR_und、SPSR_irq、SPSR_fiq)；

ARM 处理器共有 7 种运行模式，在每一种处理器模式中都有一组相应的寄存器。

2.6 ARM寄存器

2.6.1 ARM状态下的寄存器

一、各模式可访问的寄存器

在ARM状态下，任一时刻都可以访问到 16个 通用寄存器和1~2个状态寄存器。各种模式下能访问的寄存器如下图所示：

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	R0						
	R1(a2)	R1						
	R2(a3)	R2						
	R3(a4)	R3						
	R4(v1)	R4						
	R5(v2)	R5						
	R6(v3)	R6						
	R7(v4)	R7						
	R8(v5)	R8						R8_fiq *
	R9(SB,v6)	R9						R9_fiq *
	R10(SL,v7)	R10						R10_fiq *
	R11(FP,v8)	R11						R11_fiq *
	R12(IP)	R12						R12_fiq *
	R13(SP)	R13		R13_svc *	R13_abt *	R13_und *	R13_irq *	R13_fiq *
	R14(LR)	R14		R14_svc *	R14_abt *	R14_und *	R14_irq *	R14_fiq *
	R15(PC)	R15						
状态寄存器	R16(CPSR)	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器							
		用户	系统	管理	中止	未定义	中断	快中断	
通用寄存器和程序计数器	R0(a1)	R0							
	R1(a2)	R1							
	R2(a3)	R2							
	R3(a4)	R3							
	R4(v1)	R4							
	R5(v2)	R5							
	所有的37个寄存器，分成两大类： ■ 31个通用32位寄存器； ■ 6个状态寄存器。		R6						
			R7						
			R8					R8_fiq	
			R9					R9_fiq	
			R10					R10_fiq	
			R11					R11_fiq	
	R12(IP)	R12						R12_fiq	
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq	
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq	
	R15(PC)	R15							
	状态寄存器	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq	

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	R0						
	R1(a2)	R1						
	R2(a3)	R2						
	R3(a4)	R3						
	R4(v1)	R4						
	R5(v2)	R5						
	R6(v3)	R6						
	R7(v4)	R7						
	R8(v5)	R8						R8_fiq
	R9(SB,v6)	R9						R9_fiq
	R10(SL,v7)	R10						R10_fiq
	R11(FP,v8)	R11						R11_fiq
	R12(IP)	R12						R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器	R0(a1)	R0						
	R1(a2)	R1						
		R2						
		R3						
		R4						
		R5						
		R6						
		R7						
		R8						R8_fiq
		R9						R9_fiq
		R10						R10_fiq
		R11						R11_fiq
		R12						R12_fiq
	R12(r1)							
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

在汇编语言中寄存器R0~R13为保存数据或地址值的通用寄存器。它们是完全通用的寄存器，不会被体系结构作为特殊用途，并且可用于任何使用通用寄存器的指令。

2.6 ARM寄存器

二、一般通用寄存器

通用寄存器根据其分组与否可分为以下两类：

- (1) 未分组寄存器，包括 R0~R7；
- (2) 分组寄存器，包括 R8~R14。

1、未分组寄存器

在所有处理器模式下对于每一个未分组寄存器来说，指的都是同一个物理寄存器。

2、分组寄存器

它们每一个访问的物理寄存器取决于当前的处理器模式。

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器		R0						
		R1						
		R2						
		R3						
		R4						
		R5						
		R6						
		R7						
		R8						R8_fiq
	R9(SB,v6)	R9						R9_fiq
	R10(SL,v7)	R10						R10_fiq
	R11(FP,v8)	R11						R11_fiq
	R12(IP)	R12						R12_fiq
	R13(SP)	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq	
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq	
	R15(PC)	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无	SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq	

其中R0~R7为未分组的寄存器，也就是说对于任何处理器模式，这些寄存器都对应于相同的32位物理寄存器。

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
	R0(a1)	R0						
	R1(a2)	R1						
	R2(a3)	R2						
	R3(a4)	R3						
	R4(v1)	R4						
寄存器R8~R14为 分组寄存器 。它们所对应的物理寄存器取决于当前的处理器模式，几乎所有允许使用通用寄存器的指令都允许使用 分组寄存器		R5						
		R6						
		R7						
		R8						R8_fiq
		R9						R9_fiq
		R10						R10_fiq
		R11						R11_fiq
		R12						R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器						
		用户	系统	管理	中止	未定义	中断	快中断
通用寄存器和程序计数器	R0(a1)	R0						
	R1(a2)	寄存器R8~R12有两个分组的物理寄存器。一个用于除FIQ模式之外的所有寄存器模式，另一个用于FIQ模式。这样在发生FIQ中断后，可以加速FIQ的处理速度。						
	R2(a3)							
	R3(a4)							
	R4(v1)							
	R5(v2)							
	R6(v3)							
	R7(v4)							
	R8(v5)	R8						R8_fiq
	R9(SB,v6)	R9						R9_fiq
	R10(SL,v7)	R10						R10_fiq
	R11(FP,v8)	R11						R11_fiq
	R12(IP)	R12						R12_fiq
	R13(SP)	R13		R13_svc	R13_abt	R13_und	R13_irq	R13_fiq
	R14(LR)	R14		R14_svc	R14_abt	R14_und	R14_irq	R14_fiq
	R15(PC)	R15						
状态寄存器	CPSR	CPSR						
	SPSR	无		SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq

2.6 ARM寄存器

寄存器类别	寄存器在汇编中的名称	各模式下实际访问的寄存器							
		用户	系统	管理	中止	未定义	中断	快中断	
通用寄存器和程序计数器	R0(a1)	R0							
	R1(a2)	R1							
	R2(a3)	R2							
	R3(a4)	R3							
	R4(v1)	R4							
	R5(v2)	R5							
	R6(v3)	R6							
	R7(v4)	寄存器R13、R14分别有6个分组的物理寄存器。一个用于用户和系统模式，其余5个分别用于5种异常模式。							
	R8(v5)								R8_fiq
	R9(SB,v6)								R9_fiq
	R10(SL,v7)								R10_fiq
	R11(FP,v8)								R11_fiq
	R12(IP)	R12						R12_fiq	
	R13(SP)	R13	R13_svc	R13_abt	R13_und	R13_irq	R13_fiq		
	R14(LR)	R14	R14_svc	R14_abt	R14_und	R14_irq	R14_fiq		
R15(PC)	R15								
状态寄存器	CPSR	CPSR							
	SPSR	无	SPSR_svc	SPSR_abt	SPSR_und	SPSR_irq	SPSR_fiq		

2.6 ARM寄存器

2、分组寄存器

它们每一个访问的物理寄存器取决于当前的处理器模式。

- 寄存器R8 ~ R12可分为两组物理寄存器。一组用于FIQ模式，另一组用于除FIQ以外的其他模式。第1组访问R8_fiq ~ R12_fiq，允许快速中断处理。第二组访问R8_usr ~ R12_usr，没有任何指定的特殊用途。

在进入FIQ模式后就不必为保护寄存器而浪费时间，程序可以直接使用R8_fiq ~ R12_fiq来执行操作了，从而实现快速的中断处理。

2.6 ARM寄存器

- 寄存器R13 ~ R14可分为6个分组的物理寄存器。1个用于用户模式和系统模式，而其他5个分别用于svc、abt、und、irq和fiq五种异常模式。
- 如果改变处理器的模式，新模式的分组寄存器将取代原来模式的分组寄存器组。

例如，当处理器处于中断模式时，执行的指令可访问的名字仍然是R13和R14寄存器，但实际上是访问分组寄存器 R13_irq 和 R14_irq，而用户模式下的 R13_usr 和 R14_usr 不会受到任何影响，程序仍然可以正常访问R0 ~ R12寄存器。

2.6 ARM寄存器

三、堆栈指针R13

寄存器R13通常用作堆栈指针，称作SP。每种异常模式都有自己的分组R13。通常R13应当被初始化成指向给异常模式所分配的堆栈。

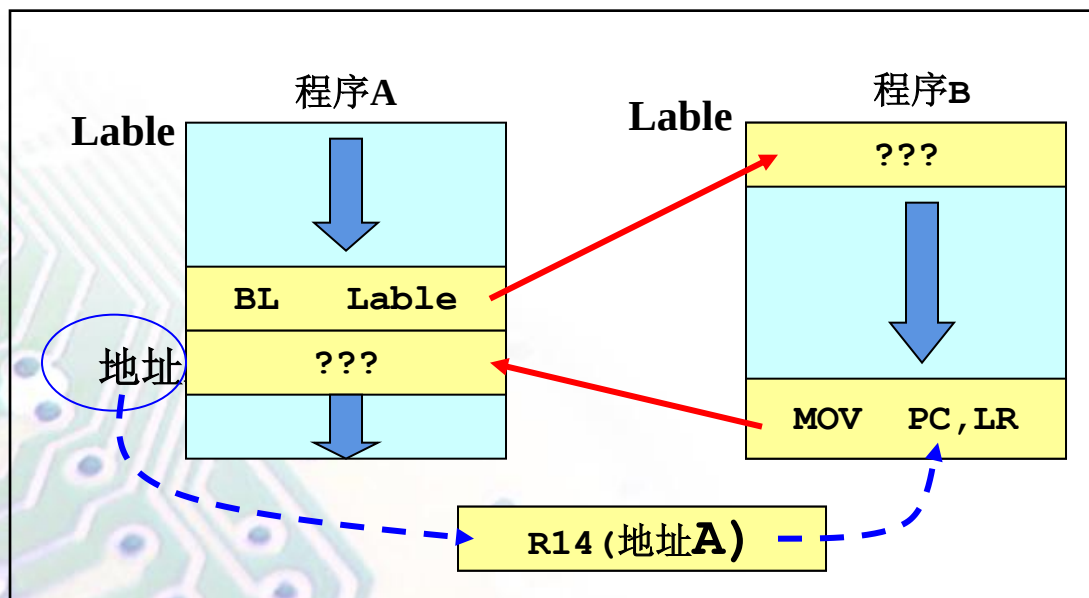
其中用户模式和系统模式共用一个，每种异常模式都有专用的R13寄存器。它们通常指向各模式所对应的专用堆栈，也就是说ARM处理器允许用户程序有6个不同的堆栈空间。

2.6 ARM寄存器

四、链接寄存器R14

寄存器R14(链接寄存器或LR)在体系结构中有两种特殊用途：

(1) 在各种模式下，R14用来保存子程序的返回地址。当一条BL或者BLX指令执行子程序调用时，R14设为子程序的返回地址。



2.6 ARM寄存器

返回时，通过复制R14中的地址值到PC中来实现子程序的返回。通常有以下2种方式：

(a) 执行下列指令之一

①MOV PC, LR ;把LR中的值写入到PC寄存器中

②BX LR ;跳转(带状态切换)到LR地址所指向的指令执行。用于从实现从一个状态返回另外一个状态并继续执行。

2.6 ARM寄存器

(b) 在子程序入口，执行R14的入栈操作：

STMFD SP!, {<registers>, LR}

；该条指令用于把<registers>和LR（R14）等多个寄存器压入堆栈，SP（R13）指向堆栈顶部。

并使用匹配的指令返回：

LDMFD SP!, {<registers>, PC}

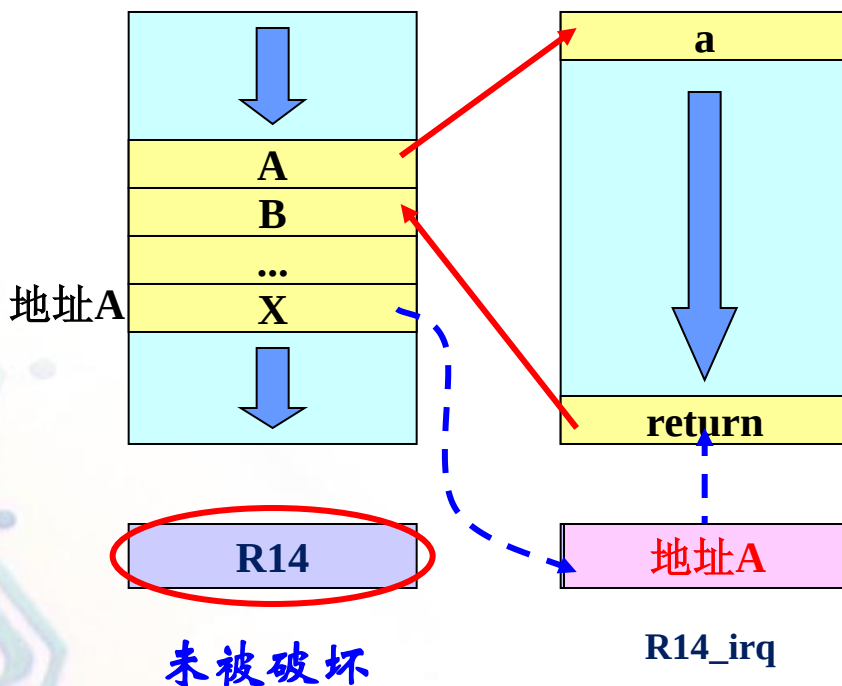
；该条指令用于把堆栈顶部多个单元，执行出栈操作，出栈的数据保存到<registers>和PC等多个寄存器中。

注：执行该操作后，原R14入栈的值，出栈后就保存到PC中了，相当于执行了把R14的值写入到PC的操作。

2.6 ARM寄存器

2. 当发生异常时，相关异常模式下的R14就设为异常返回地址。异常的返回地址与子程序的返回类似。区别在于有些异常有一个小常量的偏移。

3. IRQ服务程序A执行完毕，将R14_irq寄存器的内容减去某个常量后存入PC，返回之前被中断的程序；



2.6 ARM寄存器

五、程序计数器R15

寄存器R15用作程序计数器(PC)。在ARM状态, 位[1:0]为0, 位[31:2]保存PC。在Thumb状态, 位[0]为0, 位[31:1]保存PC。R15虽然可以像其他通用寄存器那样操作, 但对R15的使用还是有一些特殊的限制, 当违反了这些限制时, 程序的执行结果是未知的。

① 读程序计数器。指令读出的R15的值是当前指令地址加上8字节。由于ARM指令始终是字对齐的, 所以读出结果值的位[1:0]总是0(在Thumb状态下, 情况有所变化)。

2.6 ARM寄存器

- 无论处理器处于何种状态，程序计数器R15（即PC）总是指向“正在取指”的指令，而不是指向“正在执行”的指令或正在“译码”的指令。
- 一般来说，人们习惯性约定将“正在执行的指令作为参考点”，称之为当前第1条指令。
- 因此，PC总是指向第3条指令，或者说PC总是指向当前正在执行的指令地址再加2条指令的地址。

PC值 = 当前程序执行位置 + 8字节



2.6 ARM寄存器

当处理器处于Thumb状态时，每条指令长为2字节，所以PC的值为正在执行的指令地址加4字节，即：

PC值=当前程序执行位置+4字节

2.6 ARM寄存器

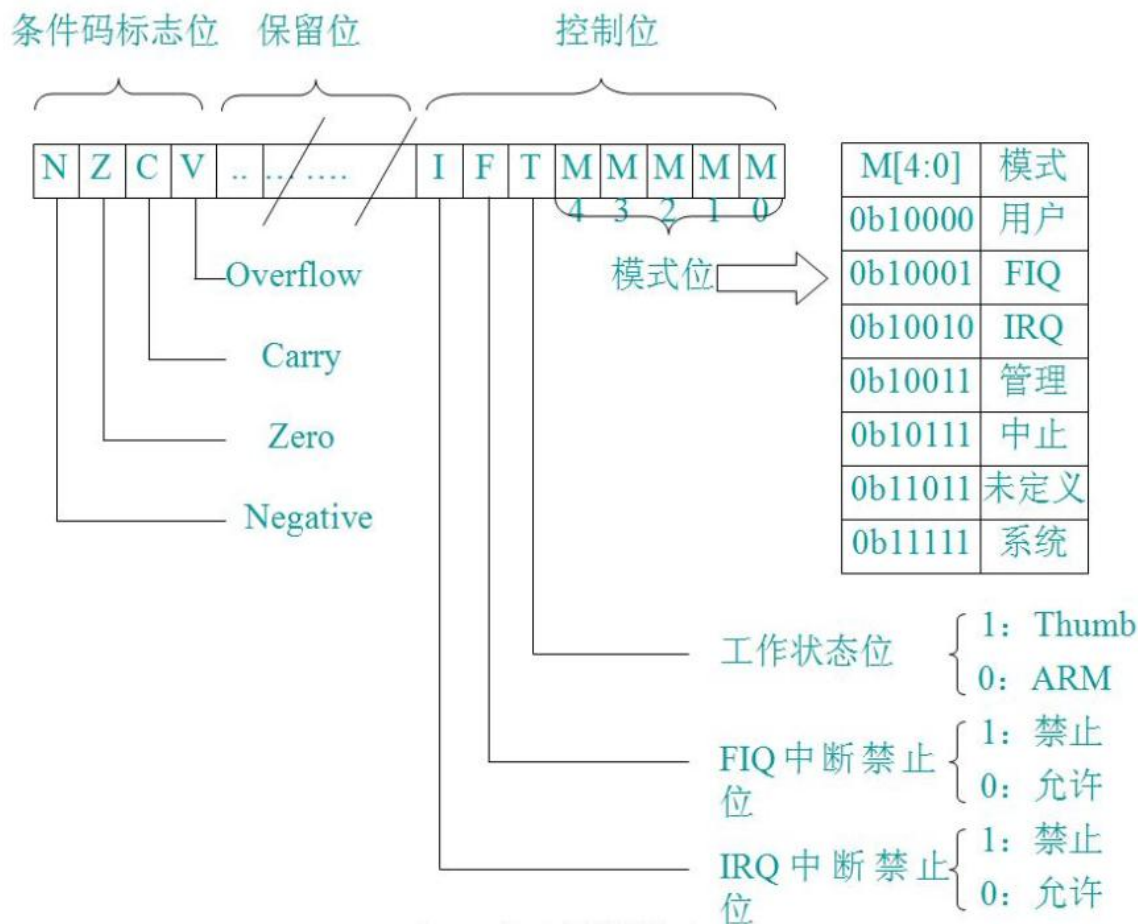
② 写程序计数器。写R15的通常结果是将写到R15中的值作为指令地址，并以此地址发生转移。由于ARM指令要求字对齐，通常希望写到R15中值的位[1:0]=0b00。

2.6 ARM寄存器

2.6.2 ARM状态下程序状态寄存器

当前程序状态寄存器（CPSR）可以在任何处理器模式下被访问，它包含下列内容：

- (1) ALU 状态标志（条件码标志位）；
- (2) 当前的处理器模式；
- (3) 中断使能标志；
- (4) 设置处理器的状态。



2.6 ARM寄存器



每一种异常器模式下，都有一个专用的物理寄存器做**备份程序状态寄存器**（SPSR： Saved Program Status Register）。当特定的异常中断发生时，这个物理寄存器负责存放当前程序状态寄存器的内容。当异常处理程序返回时，再将其内容恢复到当前程序状态寄存器。

2.6 ARM寄存器

一、条件标志位

这些条件标志位会根据程序中的**算术指令**或**逻辑指令**的执行结果进行改变，而且这些条件标志位可由大多数指令检测以决定指令是否执行。

(1) N

本位设置成当前指令执行结果的第31位。当两个由补码表示的有符号整数运算时， $N=1$ 表示结果为负数；否则结果为正数或零。

(2) Z

$Z=1$ 表示运算的结果为零，否则结果不为零。

2.6 ARM寄存器

(3) C

分 4 种情况设置 C 的方法：

- ① 在加法指令中（包括比较指令CMN），当结果产生了进位，则 $C=1$ ，表示无符号数运算发生上溢出，其它情况下 $C=0$;
- ② 在减法指令中（包括比较指令CMP），当运算中发生了借位，则 $C=0$ ，其它情况下 $C=1$;
- ③ 对于在操作数中包含移位操作的运算指令（非加/减指令），C被设置成被移位寄存器最后移出去的位;
- ④ 对于其它非加/减法运算指令，C的值通常不受影响。

2.6 ARM寄存器

(4) V

下面分两种情况讨论V的设置方法：

- ① 对于加/减运算指令，当操作数和运算结果都是以二进制的补码表示的带符号的数时，且运算结果超出了有符号运算的范围时溢出。V=1 表示符号位溢出；
- ② 对于非加/减法指令，通常不改变标志位 V 的值。

2.6 ARM寄存器

二、Q标志位（了解）

在带DSP指令扩展的ARMV5及更高版本中，Q标志位被指定用于指示增强DSP指令是否发生了溢出。

在ARMV5以前的版本及ARMV5的非E系列处理器中，Q标志位没有被定义，属于待扩展的位。

2.6 ARM寄存器

3、控制位

CPSR的低8位属于控制位。

(1) 中断禁止位（第7、6位）

I=1：IRQ被禁止；F=1：FIQ被禁止。

(2) 状态控制位（第5位）

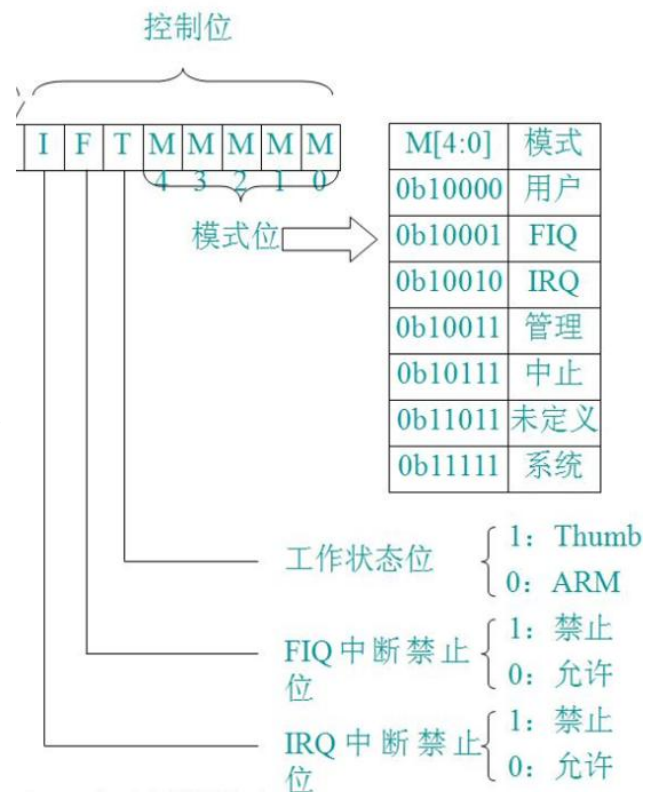
T 位是处理器的状态控制位。

T=0：处理器处于ARM状态；

T=1：处理器处于Thumb状态。

注意：绝对不要强制改变CPSR寄存器中的控制位T。

如果这样做，处理器会进入一个无法预知的状态。



2.6 ARM寄存器

(3) 模式控制位

M[4:0]	模 式	可见的Thumb状态寄存器	可见的ARM状态寄存器
10000	用户	R0~R7、SP、LR、PC、CPSR	R0~R14、PC、CPSR
10001	快速中断	R0~R7、SP_fiq、LR_fiq、PC、CPSR	R0~R7、R8_fiq、R14_fiq、PC、CPSR、SPSR_fiq
10010	中断	R0~R7、SP_irq、LR_irq、PC、CPSR	R0~R12、R13_irq、R14_irq、PC、CPSR、SPSR_irq
10011	管理	R0~R7、SP_svc、LR_svc、PC、CPSR	R0~R12、R13_svc、R14_svc、PC、CPSR、SPSR_svc
10111	中止	R0~R7、SP_abt、LR_abt、PC、CPSR	R0~R12、R13_abt、R14_abt、PC、CPSR、SPSR_abt
11011	未定义	R0~R7、SP_und、LR_und、PC、CPSR	R0~R12、R13_und、R14_und、PC、CPSR、SPSR_und
11111	系统	R0~R7、SP、LR、PC、CPSR	R0~R14、PC、CPSR

注意：如果将非法值写入M[4:0]中，处理器将进入一个无法恢复的模式。



2.6 ARM寄存器

注意：

- 通过程序修改CPSR可以进入异常。除此之外，也可以在内核对异常或者中断响应时由硬件切换到异常模式。

例：从系统模式切换到管理模式



MSR CPSR_c, #D3 ;CPSR的控制位为#D3 (11010011) ,
切换到管理模式。

- 用户模式与系统模式不能由异常进入，也就是说要想进入系统模式，必须通过修改CPSR才能实现。

MSR CPSR_c, #DF ;CPSR的控制位为#DF (11011111) ,
切换到系统模式（比如从管理模式到系统模式）。

2.7 ARM存储系统

2.7.1 ARM存储方法

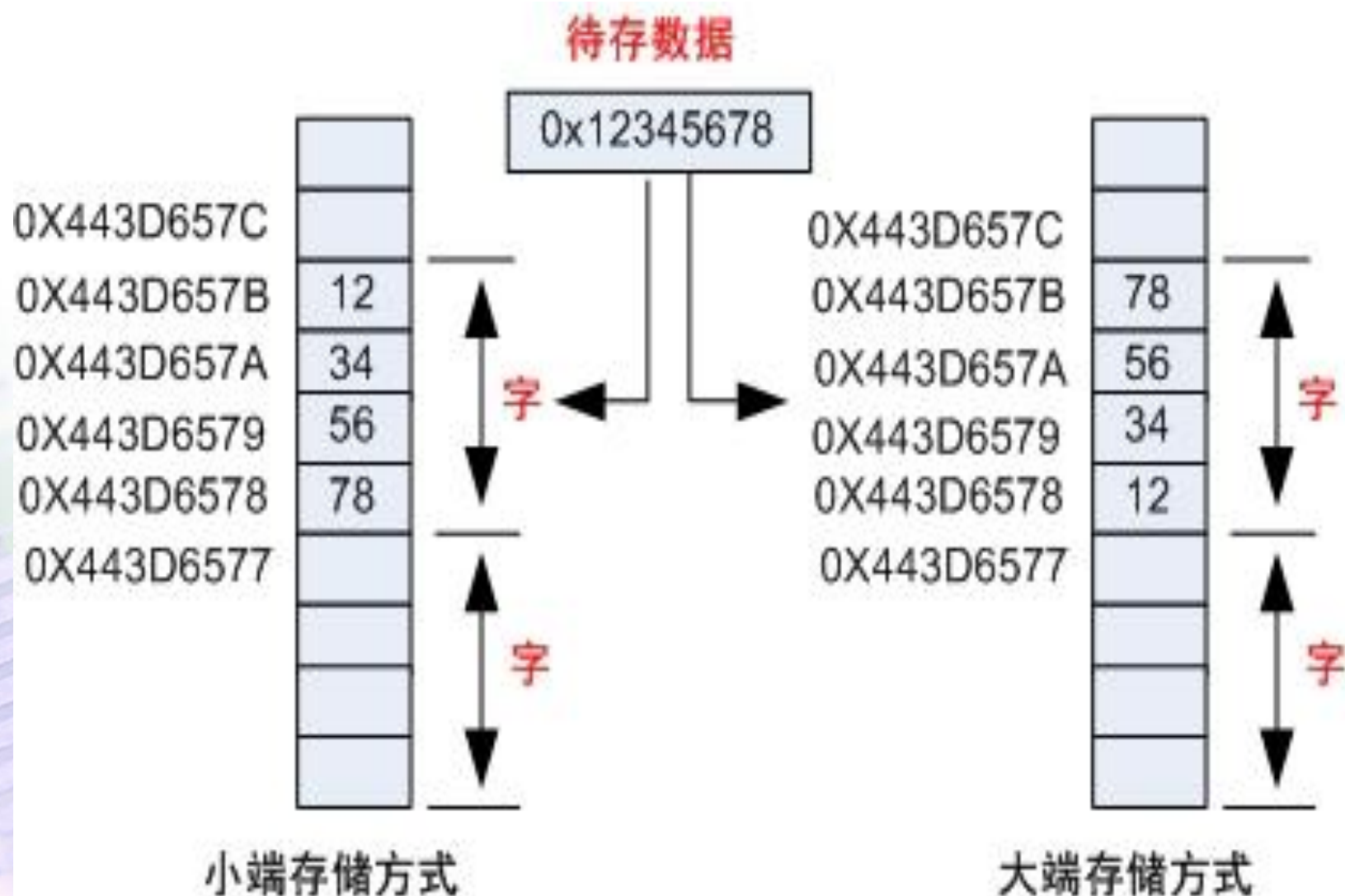
ARM处理器外部采用冯·诺依曼（von Neumann）结构，指令和数据共用一条32位数据总线，只有装载、存储和交换指令可访问存储器中的数据。

ARM处理器将存储器看作是一个从0开始的线性递增的字节集合，其中每个数组元素（字节）都是可以寻址的。

ARM支持**大端模式**（big-endian）和**小端模式**（little-endian）两种内存模式。

在大端模式下，一个字的高地址单元放的是数据的低位；而在小端模式下，数据的低位放在内存中低地址单元中。

2.7 ARM存储系统



2.7 ARM存储系统

一个基于ARM的**实际芯片**可能只支持小端存储器格式，也可能只支持大端存储器格式，还可能两者都支持。

ARM指令集不包含任何直接选择大小端存储器格式的指令，但是一个同时支持大小端存储器格式基于ARM的芯片可以通过硬件配置（一般使用芯片的引脚来配置）来匹配存储器系统所使用的规则。如果芯片有一个标准系统控制的协处理器（例如ARM920T的CP15协处理器），系统控制协处理器寄存器1的bit7可用于改变配置。

注意：对于S3C2440是通过软件来指定存储器格式的（通过设置CP15协处理器协处理器寄存器1的bit7来实现），缺省为小端格式。

2.7 ARM存储系统

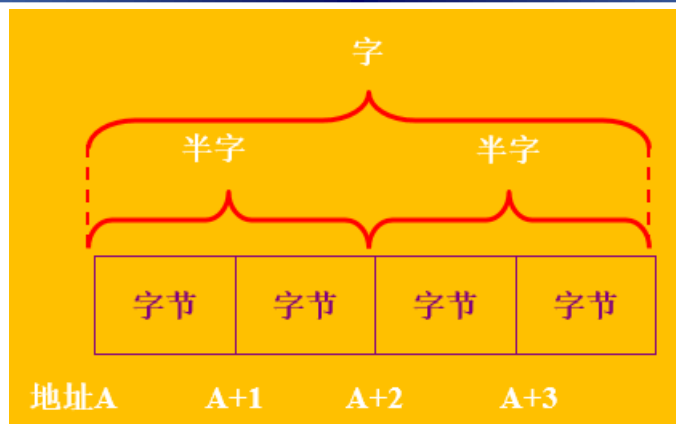
2.7.2 ARM体系的存储空间

ARM9使用 2^{32} 个8位字节地址空间，字节地址的排列从 $0 \sim 2^{32}-1$ 。

地址空间也可以看作是包含 2^{30} 个32位字，地址以字为单位进行分配。也就是将地址除以4，地址为A的字包含4个字节，地址分别为A、A+1、A+2和A+3。

与此同时，地址空间还可被看作包含 2^{31} 个16位半字，地址按照半字进行分配。地址为A的半字包含2个字节，地址分别为A和A+1。

2.7 ARM存储系统



通常地址计算通过普通的整数加减指令来实现。这意味着如果地址向上或向下溢出地址空间，通常会发生翻转。如果地址的计算没有发生翻转，那么结果仍然位于 $0 \sim 2^{32}-1$ 范围内。

程序应保证穿过地址0xFFFF FFFF的向前转移和穿过地址0x0000 0000的向后转移的情况都不发生。

2.7 ARM存储系统

2.7.3 存储器结构

ARM处理器有的带有指令cache和数据cache，但不带有片内RAM和片内ROM。

系统所需的RAM和ROM（包括Flash）都通过总线外接。

有的ARM片内还带有存储器管理单元MMU（Memory Management Unit）。

2.7 ARM存储系统

2.7.4 存储器映射I/O

基于ARM内核的芯片具有许多的外设，这些外设访问的标准方法是使用存储器映射的I/O（存储器统一编址），为外设的每个寄存器都分配一个地址。通常，从这些地址装载数据用于读入，向这些地址保存数据用于输出。有些地址的装载和保存用于外设的控制功能，而不是输入或输出功能。

注意：存储器映射的I/O位置的操作不同于正常的存储器位置的操作。通常，把存储器映射的I/O位置标记为无高速缓存和无缓冲区，以避免处理器把这部分地址当普通内存，调入到cache中，进行缓存操作。

2.7 ARM存储系统

2.7.5 Flash组织形式

目前几乎所有MCU的程序都存储在Flash中。Flash分为片内和片外两种。片外Flash又有以下种类。

(1) NOR Flash

NOR Flash读取速度快，但是擦除和写入速度慢。

(2) NAND Flash

NAND Flash读取速度比NOR Flash慢，但擦除和写入速度很快。

2.7 ARM存储系统

2.7.5 RAM的组织形式

芯片自带的RAM不够用时需要外加，外加的RAM一般是SDRAM和DDR RAM，极少数使用SRAM，因为SRAM的价格相对较贵，但SRAM速度要比动态RAM快。

- ① SRAM。只需要在IDE中设置好地址。
- ② SDRAM/DDR RAM。不仅要在IDE中设置好起始地址，还要在程序中做初始化操作后才能使用。如果使用仿真器，那么在仿真运行程序前，要执行一个初始化脚本文件，或者执行一系列命令对SDRAM和DDR RAM进行初始化，否则会运行出错。

2.8 中断和异常的基本概念（不考）

中断是主机与外设进行数据通信的重要机制，它负责处理处理器**外部的事件**；**异常**实质上也是一种中断，只不过它主要负责处理处理器**内部事件**。

计算机通常是用中断来处理外中断和内部异常，因此在ARM技术中，将二者统称为异常。

2.8 中断和异常的基本概念

2.8.1 中断和异常

1、什么叫中断

当处理器遇有外部设备发生 “紧急事件” 需要它来处理时，它就必须停下 “手头上的工作” 先去处理这个 “紧急事件”。处理器的这种工作过程，或者这种工作状态就叫做中断。

2、什么叫中断请求

当外部设备有紧急事件需要处理器进行处理时，外部设备必须向处理器发送一个 电信号（脉冲或电平）来表示有事件需要处理器来处理。这个信号叫做 中断请求信号，或称中断请求。

2.8 中断和异常的基本概念

3、什么叫中断源

发出中断请求信号的外部设备或事件就叫做**中断源**。

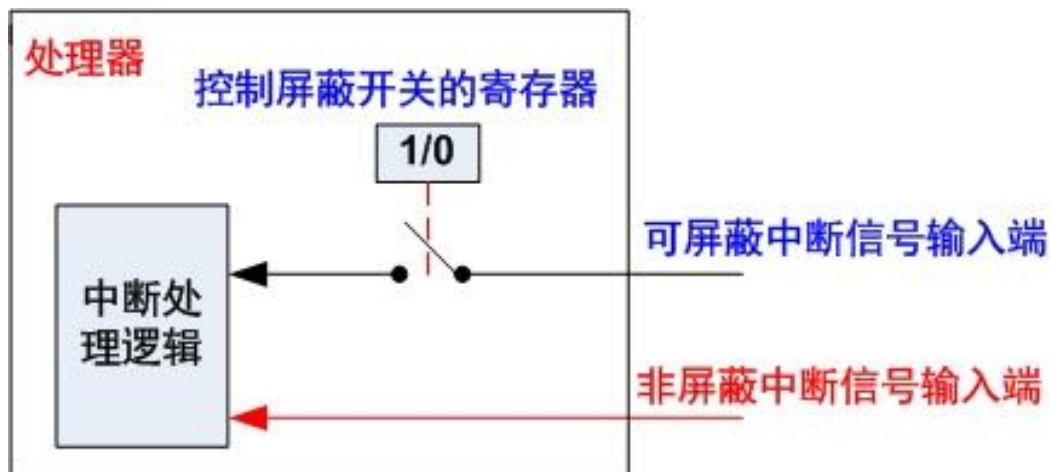
4、什么叫异常

由内部事件引起的中断叫做异常。

2.8 中断和异常的基本概念

2.8.2 中断请求信号的屏蔽

处理器中用来屏蔽中断的寄存器和开关如下：



1、可屏蔽中断

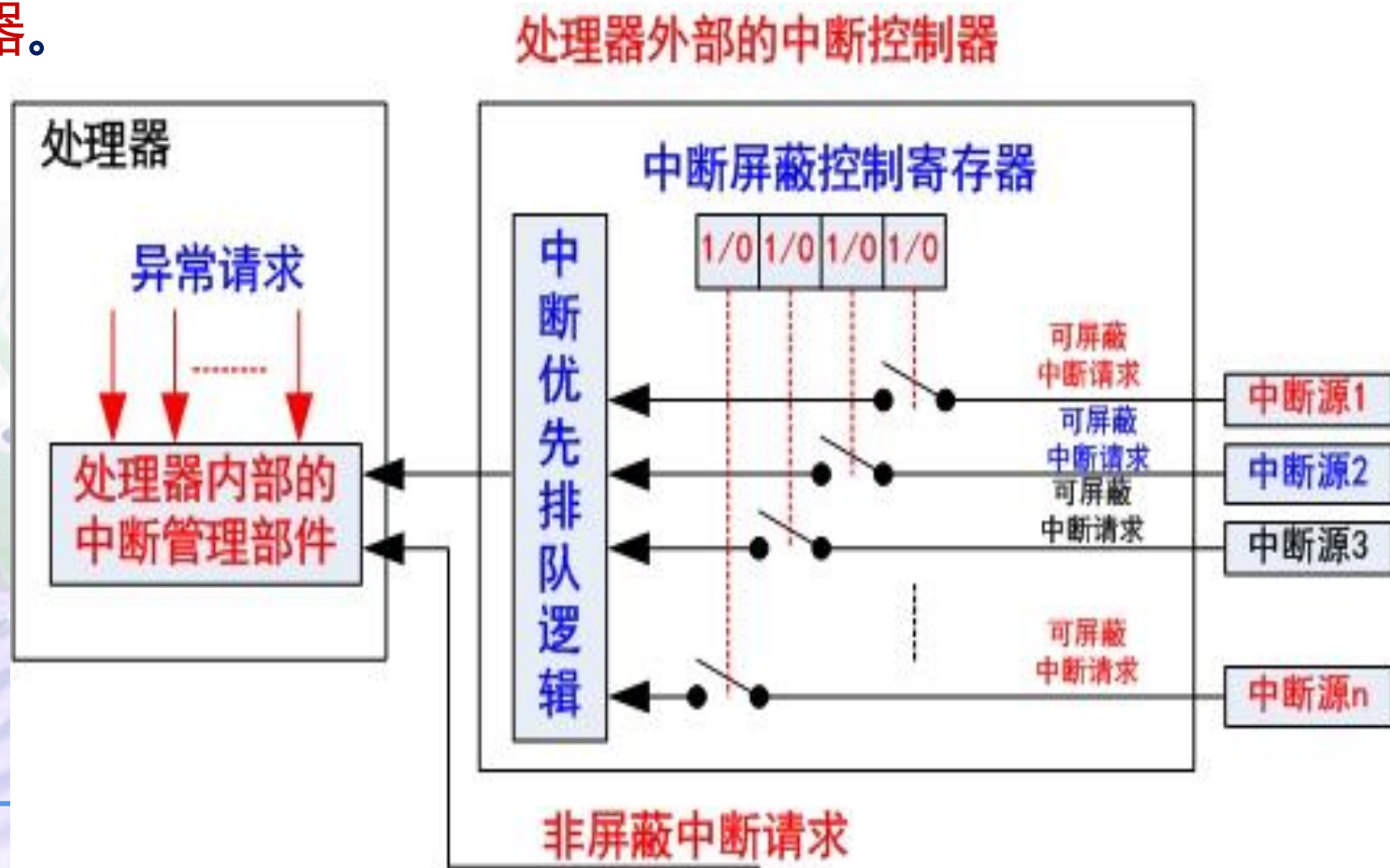
人们把带有开关，能阻止中断请求的中断输入端叫做可屏蔽中断信号输入端。这类中断叫可屏蔽中断。

2、非屏蔽中断

人们把不带开关，不能阻止中断请求的中断输入端叫做非屏蔽中断信号输入端。这类中断叫非屏蔽中断。

2.8 中断和异常的基本概念

为了对处理器可以接收中断源的数目进行扩充及对中断进行管理，在中断源和处理器之间还配有如下图所示的中断控制器。



2.8 中断和异常的基本概念

2.8.3 中断优先级及中断嵌套

1、中断优先级

处理器通常只有一个可屏蔽中断请求输入端。对于具有多个中断源的系统来说，当有两个或两个以上中断源同时发生中断请求时就会出现所谓的竞争。

竞争可以通过优先级的方法来处理，具体实现方法有两种：硬件实现方法和软件实现方法。

2.8 中断和异常的基本概念

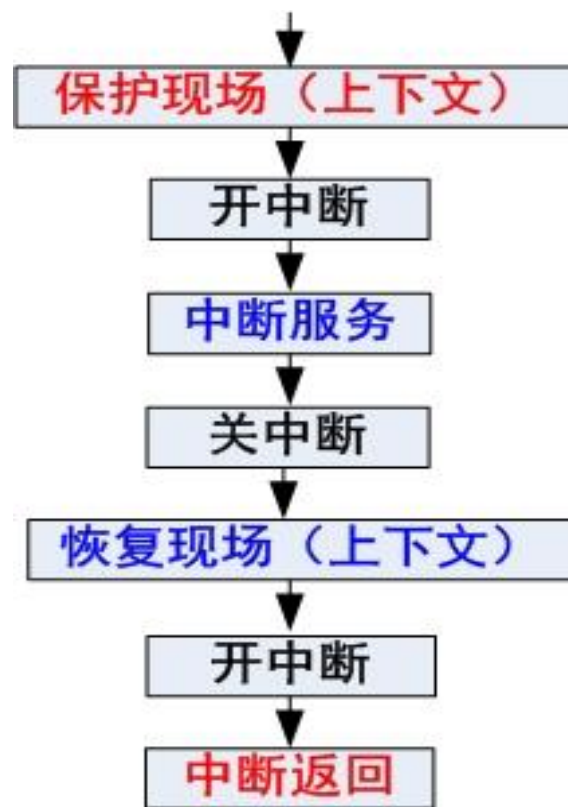
2.8.4 中断服务程序

用来处理中断事件的程序叫做**中断服务程序**。

中断服务程序的一般结构如右所示：

中断服务程序与普通子程序的重要差别在于：

中断服务程序要对**中断嵌套**进行必要的管理。即中断服务程序要根据需要，对程序状态寄存器中的中断允许标志进行相应的设置。



2.8 中断和异常的基本概念

2.8.5 中断向量和中断向量表

为了与普通子程序的首地址进行区分，中断服务程序的首地址（入口地址）通常被叫做**中断向量**，或**中断矢量**。

以后还会看到，凡是能直接或间接指向中断服务程序的都叫**中断向量**。

在处理器收到中断请求之后，它们都需要获得中断服务程序首地址——中断向量。

所有的中断向量都按一定规律存放在一个**固定的存储区域**，这个集中存放了中断向量或与中断向量相关信息的存储区域就叫做**中断向量表**。

2.8 中断和异常的基本概念

2.8.6 中断的处理过程

1、处理器响应中断的条件

处理器响应中断的条件主要有以下几个：

- (1) 处理器程序状态寄存器的中断屏蔽标志处于非屏蔽状态；
- (2) 没有更高级的中断请求正在响应或正在发出、正挂起；
- (3) 处理器在现行指令执行结束后。

2.8 中断和异常的基本概念

2、中断的处理过程

当中断请求发生且满足上述条件时，计算机系统就会响应中断请求，并自动将被中断程序的下一条指令地址（断点地址）保存和关闭中断；接下来便将自中断向量表查询得到的与该中断源对应的中断向量送入PC，并转去执行中断服务程序。

当执行到中断服务程序末尾时，执行中断返回指令或跳转指令，把保存的断点地址送回 PC，以在断点处继续执行被中断的程序。

2.9 ARM的中断（异常）

只要正常的程序流程被暂时中止，ARM处理器就进入异常模式。

ARM异常与中断不做严格意义上的区别。

ARM处理器可以响应的中断（异常）有：中断、快中断、复位中断、软中断异常、预取指令中止异常、数据中止异常和未定义指令异常 7 种。

2.9 ARM的中断（异常）

2.9.1 ARM的中断（异常）向量表

ARM的中断向量表内存放的是响应异常和中断的转移指令而不是中断向量地址。

在ARM处理器中，当异常发生时，完成当前指令后跳转到相应的异常中断处理程序入口执行异常中断处理。异常处理完毕后返回原来的程序断点继续执行原来的程序。

2.9 ARM的中断（异常）

1、低端和高端向量表

ARM有低端和高端两种向量表，用户可以根据需要选用其中一种，如下所示：

	4字节			
0x0000001C				
0x00000018				
0x00000014				
0x00000010				
0x0000000C				
0x00000008				
0x00000004				
0x00000000				

低位方式的中断（异常）向量表

	4字节			
0xFFFF001C				
0xFFFF0018				
0xFFFF0014				
0xFFFF0010				
0xFFFF000C				
0xFFFF0008				
0xFFFF0004				
0xFFFF0000				

高位方式的中断（异常）向量表

2.9 ARM的中断（异常）

ARM中断（异常）的各个向量在向量表中的分配如下：

中断（异常）	向量在低端向量表的地址	向量在高端向量表的地址
复位（RESET）	0x00000000	0xFFFF0000
未定义指令（UNDEF）	0x00000004	0xFFFF0004
软中断（SWI）	0x00000008	0xFFFF0008
预取指令中止（PABT）	0x0000000C	0xFFFF000C
数据中止（DABT）	0x00000010	0xFFFF0010
保留	0x00000014	0xFFFF0014
中断（IRQ）	0x00000018	0xFFFF0018
快中断（FIQ）	0x0000001C	0xFFFF001C

高端向量是ARM架构可选配置，可以通过硬件外部输入管脚来配置是低端向量还是高端向量，不能通过指令来改变向量的位置。但可通过协处理器CP15配置。

2.9 ARM的中断（异常）

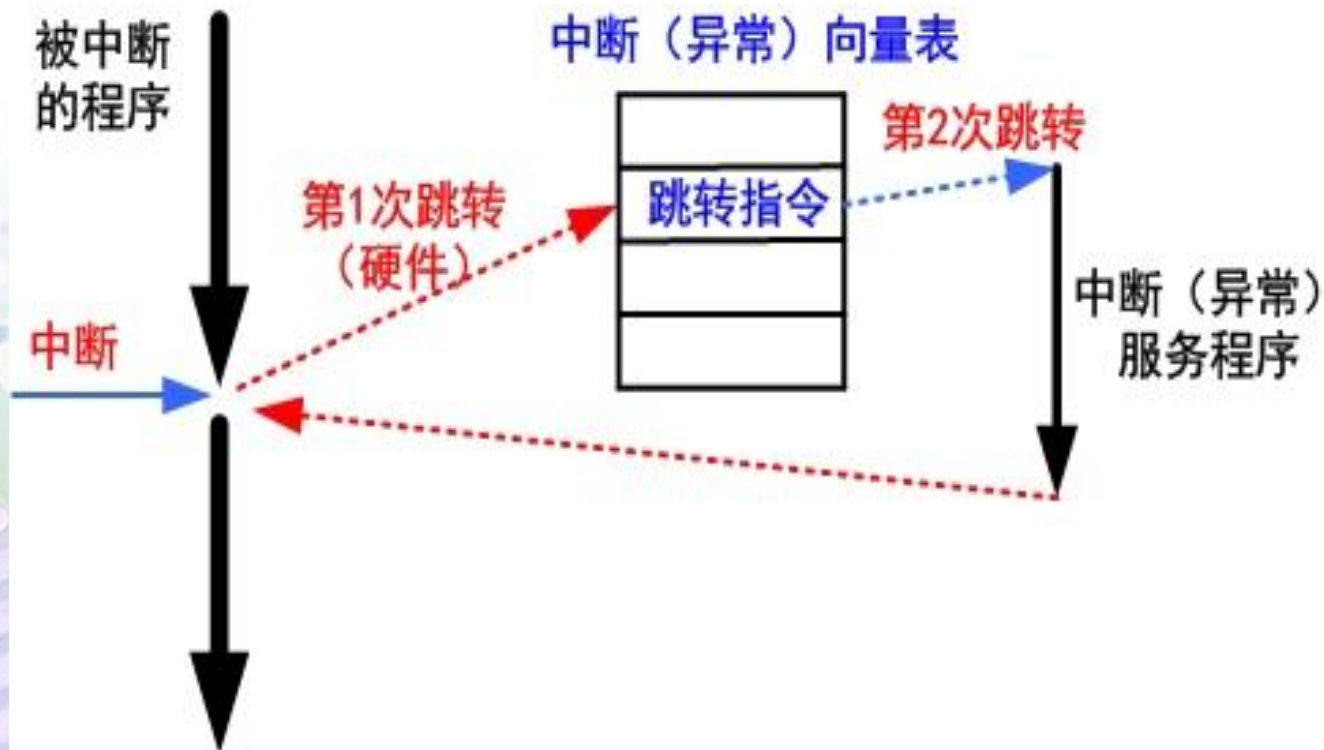
ARM中断（异常）的各个向量在向量表中的分配如下：

地址	异常类型	进入时的模式	进入时I的状态	进入时F的状态
0x0000 0000	复位	管理	禁止	禁止
0x0000 0004	未定义指令	未定义	I	F
0x0000 0008	软件中断(SWI)	管理	禁止	F
0x0000 000C	预取中止(指令)	中止	I	F
0x0000 0010	数据中止	中止	I	F
0x0000 0014	保留	保留	—	—
0x0000 0018	IRQ	中断	禁止	F
0x0000 001C	FIQ	快中断	禁止	禁止

注：表中的I和F表示不对该位有影响，保留原来的值。

2.9 ARM的中断（异常）

处理器在响应中断（异常后），可以通过**两次跳转**转移到中断（异常）服务程序。两次跳转的示意图如下：



2.9 ARM的中断（异常）

2.9.2 ARM异常响应

一、进入异常

在异常发生后，ARM内核会作以下工作：

1. 在适当的LR中保存断点的地址，当异常入口来自：
无论是ARM状态还是Thumb状态，ARM都将当前PC寄存器的值（运行指令地址加4或加8）复制(取决于异常的类型) 到LR中；
2. 把当前程序状态寄存器（CPSR）中的内容保存到模式私有寄存器SPSR中；

2.9 ARM的中断（异常）

3. 将寄存器CPSR中的MODE域设置为中断（异常）应进入的运行模式；
4. 对CPSR的I位和F位进行相应的设置，以防止再次响应同一个中断请求。
5. 强制PC从相关的异常向量处取指，即到中断向量表中获取中断向量，转向用户所编写的中断（异常）服务程序。

注：异常总是在ARM状态中进行处理。当处理器处于Thumb状态时发生了异常，在异常向量地址装入PC时，会自动切换到ARM状态。

2.9 ARM的中断（异常）

ARM处理器对异常的响应过程可以用伪代码描述如下：

R14_<exception_mode>=return link (PC的值)

SPSR_<exception_mode>=CPSR

CPSR [4: 0] =exception mode number

CPSR [5] =0 /*当运行于ARM状态时*/

CPSR [6] =1 /*禁止新的IRQ中断*/

if <exception-mode>=Reset or FIQ then

 CPSR [7] =1 /*当Reset 或FIQ异常中断时*/

 /*禁止新 的FIQ中断*/

PC=exception vector address

2.9 ARM的中断（异常）

二、异常返回

当异常结束时，异常处理程序必须做如下处理：

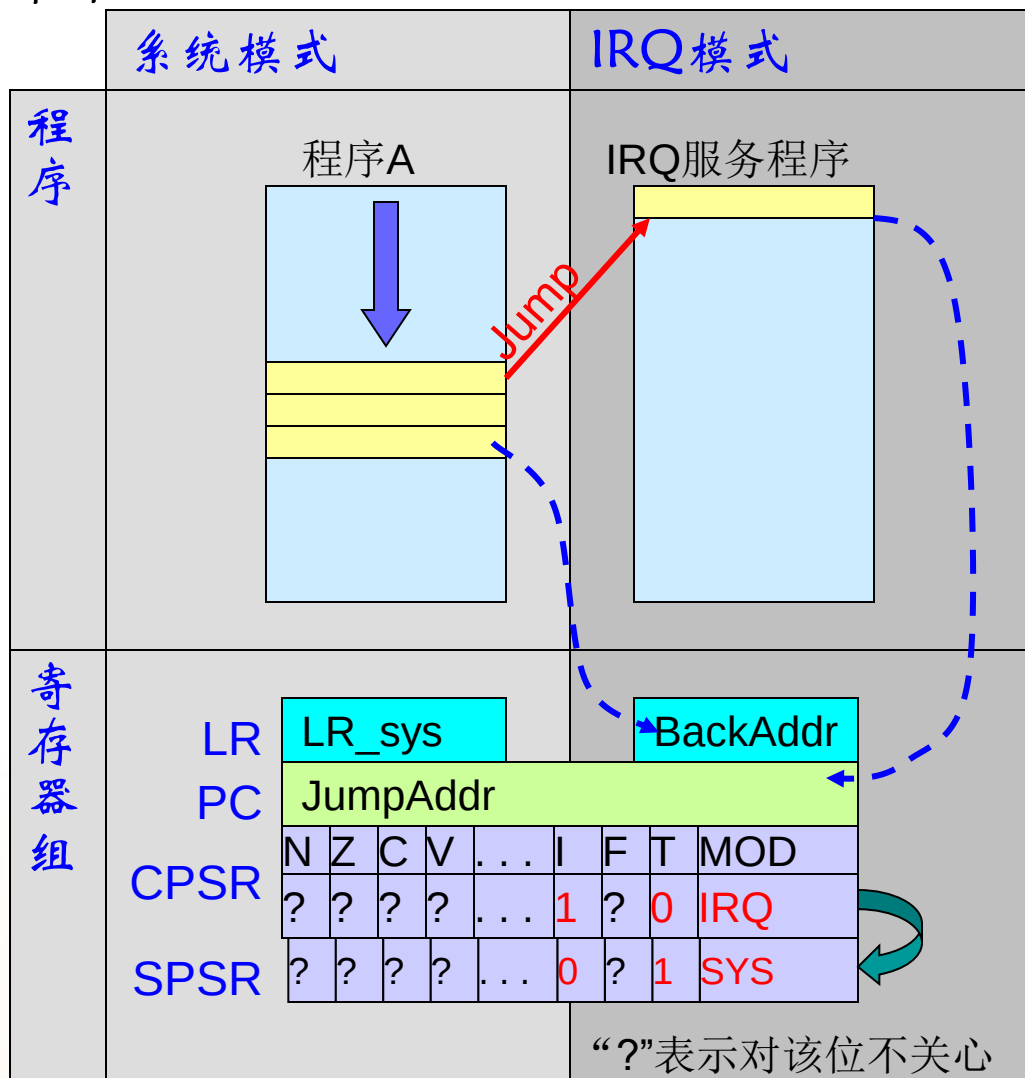
1. 将SPSR的值复制回CPSR
2. 将LR中的值减去偏移量后存入PC，偏移量根据异常的类型而有所不同。

注：恢复CPSR的动作会将T、F和I位自动恢复为异常发生前的值。

2.9 ARM的中断（异常）

图示进入异常过程

2. 用户程序运行时发生IRQ中断在系统模式完成以下运行操作
- 将当前寄存器内容存入允许IRQ模式的SPSR寄存器
 - 置位I位（禁止IRQ中断）
 - 清零T位（进入ARM状态）
 - 设置MOD位，切换处理器模式至IRQ模式
 - 将当前PC的内容存入IRQ模式的LR寄存器
 - 将向量地址存入PC，实现跳转

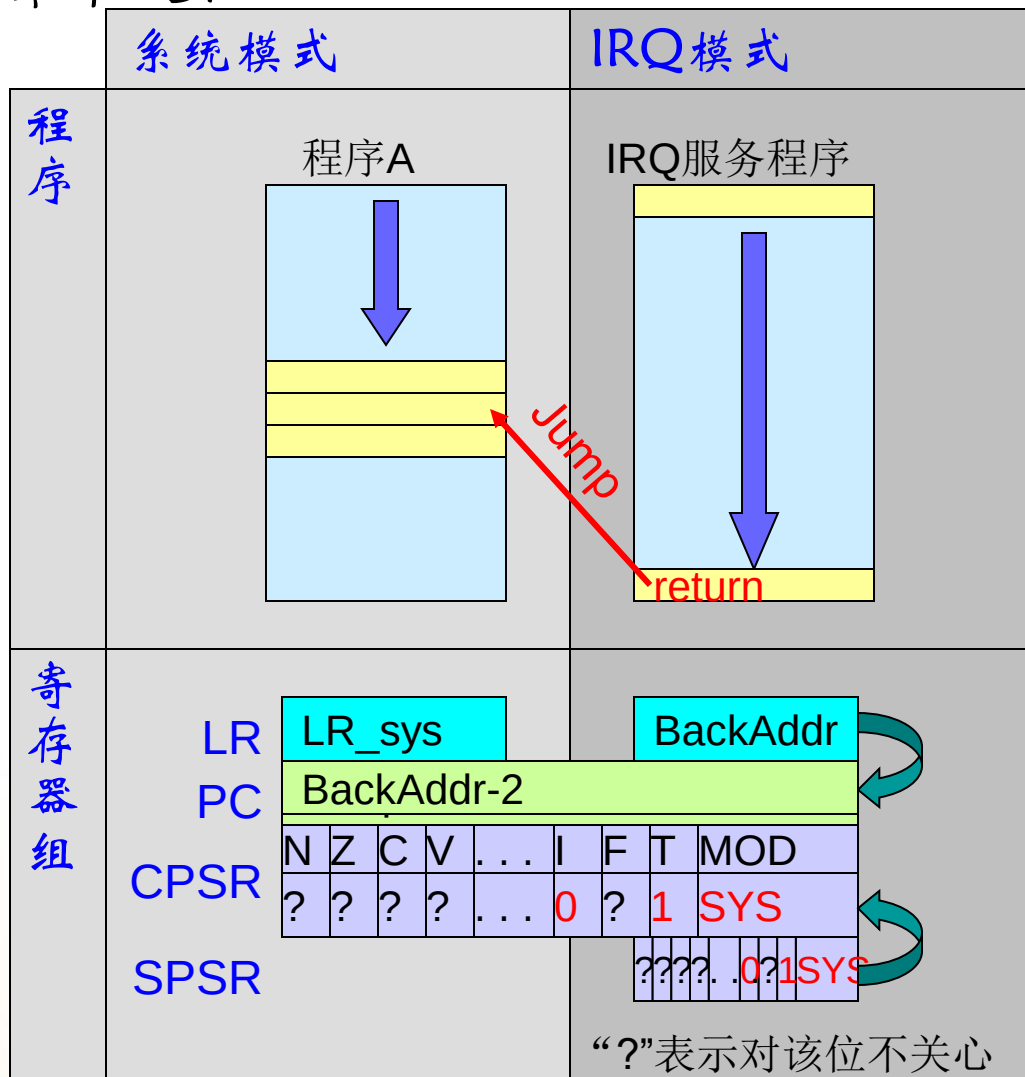


2.9 ARM的中断（异常）

图示退出异常过程

在异常处理结束后，异常处理程序完成以下动作：

- 将SPSR寄存器的值复制回CPSR寄存器；
- 将LR寄存的值减去一个常量后复制到PC寄存器，跳转到被中断的用户程序。



2.9 ARM的中断（异常）

四、ARM异常进入/退出

ARM处理器进入异常时R14所保存的值及退出时推荐指令：

异常类型	返回指令	之前的状态		备注
		ARM R14_x	Thumb R14_x	
BL	MOV PC,R14	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	此处 $\text{Adr}_{\text{执}}$ 为BL，SWI，未定义的指令取指或预取指中止指令的地址
软件中断 SWI	MOV ^S PC,R14_svc	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	
未定义的指令	MOV ^S PC,R14_und	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	
预取指中止	SUB ^S PC,R14_abt,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 4$	PC+2	
快中断	SUB ^S PC,R14_fiq,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+4	此处 $\text{Adr}_{\text{执}}$ 为FIQ或IRQ产生时，正在执行指令的地址
中断	SUB ^S PC,R14_irq,#4	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+4	
数据中止	SUB ^S PC,R14_abt,#8	$PC_{\text{寄}} = \text{Adr}_{\text{执}} + 8$	PC+4	此处 $\text{Adr}_{\text{执}}$ 为产生数据中止的装载或保存指令的地址。
复位	无	—	—	复位时保存在R14_svc中的值不可预知。

2.9 ARM的中断（异常）

如果异常处理程序已经把返回地址拷贝到堆栈，那么可以使用堆栈指令来恢复用户寄存器并实现返回。

中断处理的开始部分和退出部分的代码(程序框架)如下(以IRQ中断为例)：

```
SUB    LR,LR,#4           ;计算返回地址 -4
STMFD  SP!,{R0-R3,LR}     ;保存使用到的寄存器
...
LDMFD  SP!,{R0-R3,PC}^    ;中断返回
```

注意：中断返回指令的寄存器列表（其中必须包括PC）后的“^”符号表示这是一条特殊形式的指令。它指示在从存储器中装载PC的同时（PC是最后恢复的），CPSR也得到恢复。

2.9 ARM的中断（异常）

2.9.3 ARM异常描述

ARM按事件的紧急程度为每个中断（异常）都定义了一个固定的优先级别。当多个异常同时发生时，一个固定的优先级系统决定它们被处理的顺序：

异常类型	优先级	 优先级降低
复位	1（最高优先级）	
数据中止	2	
FIQ	3	
IRQ	4	
预取中止	5	
未定义指令	6	
SWI	7（最低优先级）	

2.9 ARM的中断（异常）

一、复位中断

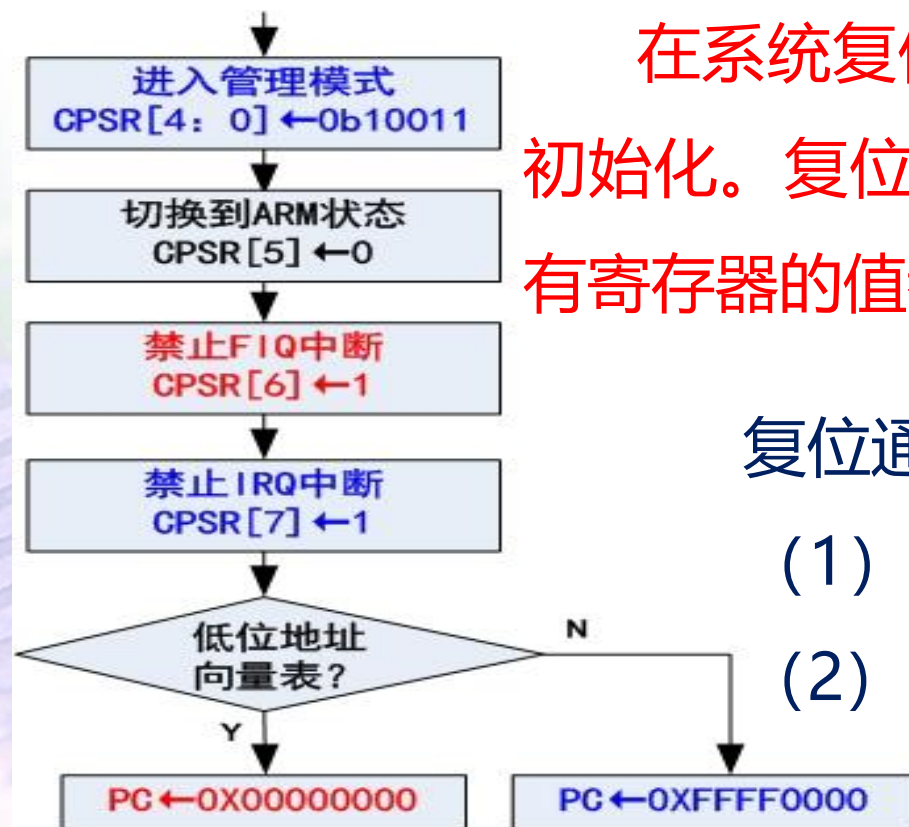
当nRESET信号被拉低时（一般外部复位引脚电平的变化和芯片其他复位源会改变这个内核信号），ARM处理器**放弃正在执行的指令**。

当nRESET信号再次变为高电平时，ARM处理器执行以下操作：

- 强制M[4 : 0]变为b100011，系统进入**管理模式**；
- 将CPSR中的控制位I和F置位，**IRQ和FIQ中断禁止**；
- 将CPSR中的控制位T清零，处理器处于**ARM状态**；

2.9 ARM的中断（异常）

- 强制PC从地址0x00开始对下一条指令进行取指；
- 返回到ARM状态并恢复执行。



在系统复位后，进入管理模式对系统初始化。复位后，除PC和CPSR之外的所有寄存器的值都是随机的。

复位通常在两种情况下发生：

- (1) 系统初始运行时的正常上电；
- (2) 由程序引起的复位。

2.9 ARM的中断（异常）

二、中断请求异常IRQ

只有当CPSR中相应的中断屏蔽被清除时，才可能发生中断请求（IRQ）异常，IRQ异常是一个由nIRQ输入端的低电平所产生的正常中断。

当一个IRQ异常中断发生时，内核切换到中断模式，表明产生了中断。

2.9 ARM的中断（异常）

内核自动地做如下处理：

- ① 将异常处理程序的返回地址（PC寄存器中的值）保存到异常模式下的R14（R14_irq或者LR）中，即“中断返回地址 + 4”。
- ② 用户模式的CPSR被保存到新的IRQ中断异常模式SPSR_irq中。

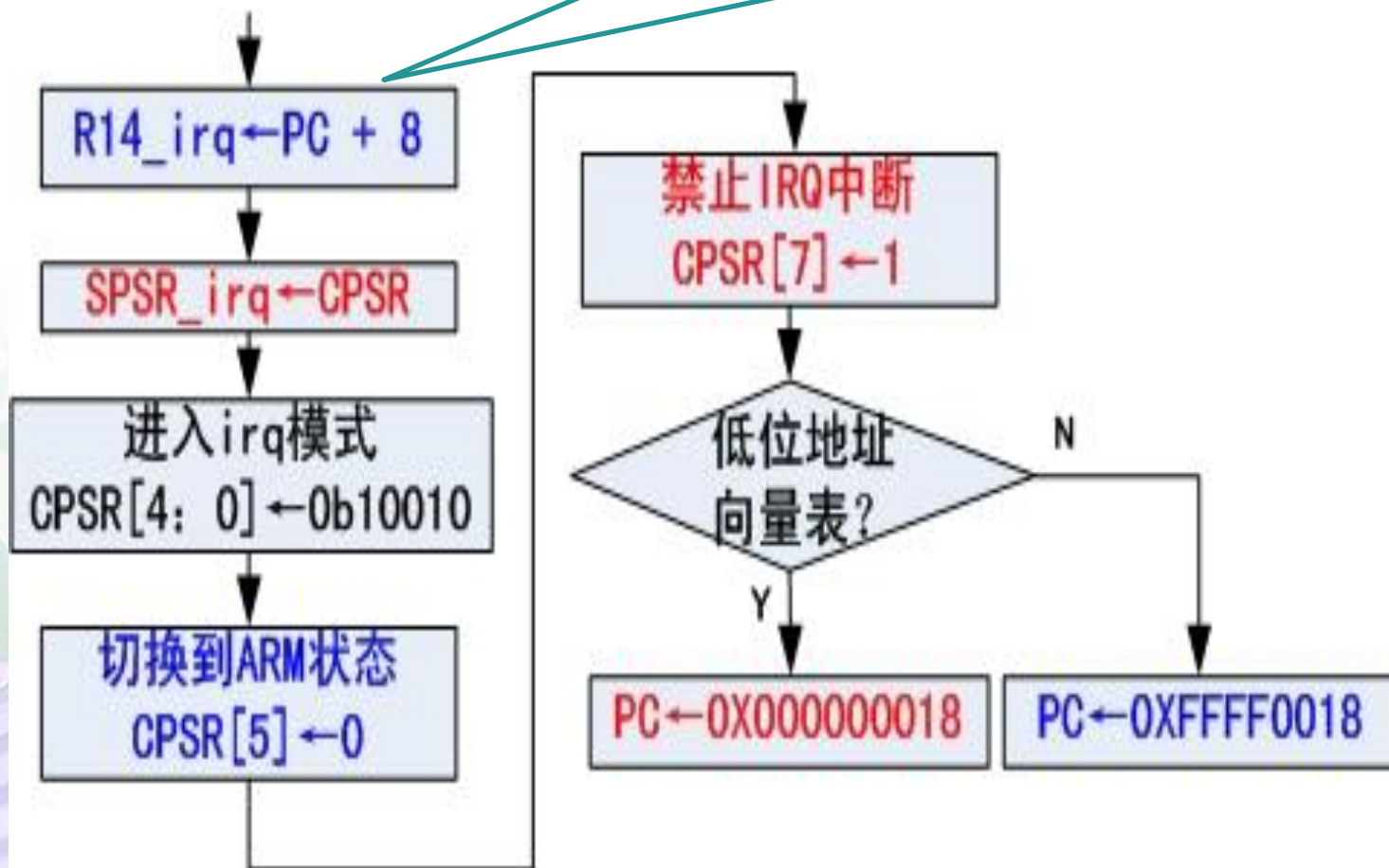
2.9 ARM的中断（异常）

- ③ 修改CPSR。将I位置1，禁止新的IRQ中断产生，但是不限制FIQ中断的发生（F位保持原有状态）。清零T标志位，CPU进入ARM状态。修改模式位，设置为IRQ模式，此时用户模式下的R13和R14将不可操作，而IRQ模式下的R13和R14变为可操作，即R13_irq保存IRQ中断模式的堆栈指针，R14_irq保存了“IRQ中断返回地址+4”。
- ④ 设置IRQ模式下的PC为IRQ异常处理程序的中断入口向量地址，在IRQ模式下，该向量地址为0x0000 0018。

2.9 ARM的中断（异常）

IRQ响应过程：

这里的PC指向的是产生中断时，正在运行的指令。PC+8就是PC寄存器里的值



2.9 ARM的中断（异常）

返回时，可以通过“SUBS PC, R14, #4”指令返回。

注意：

- ① 实际上在中断模式下执行的是“SUBS PC, R14, #4”指令，同时在SUB指令尾部有一个S，并且PC是目的寄存器，所以CPSR将自动从SPSR寄存器中恢复。
- ② 如果用户需要嵌套IRQ中断，那么必须在中断服务程序中重新使能IRQ中断，并将R14压入R13所指向的IRQ堆栈之中以预先保留返回地址。程序应当首先通过压入IRQ堆栈来预先保留ISR（中断服务子程序）将会使用的R0～R12的值，这时就可以对异常进行处理了。

2.9 ARM的中断（异常）

三、快速中断请求异常FIQ

有些嵌入式系统的应用对实时性要求比较高，需要足够快的中断响应速度，比如数据转移或通道处理。ARM在设计上充分地考虑了嵌入式系统的这一特点，在IRQ异常之外还设计了一种快速中断请求（FIQ）异常，并在硬件结构和资源分配上给予了足够的支持。

2.9 ARM的中断（异常）

为减少延时，ARM在快中断中采取了两个措施：

(1) 专门为快中断配置了较多的私有寄存器，从而可使中断服务程序有足够的寄存器来使用，而不必与被中断服务程序使用同一组寄存器，这样就免去了因寄存器冲突而必需的保护及恢复现场工作。

(2) ARM把FIQ的中断向量放在了中断（异常）向量表末尾0X0000001C处，因此它后面没有其它中断向量，允许用户将中断服务程序直接放在这里。

2.9 ARM的中断（异常）

只有当CPSR中相应的F位被清零时，才可能发生FIQ异常。

将ARM内核的nFIQ信号拉低，可实现外部产生FIQ，FIQ异常是优先级最高的外部中断。内核进入FIQ处理程序之后，FIQ、IRQ同时禁止任何外部中断源再次发生中断，除非在软件中重新使能IRQ、FIQ请求。

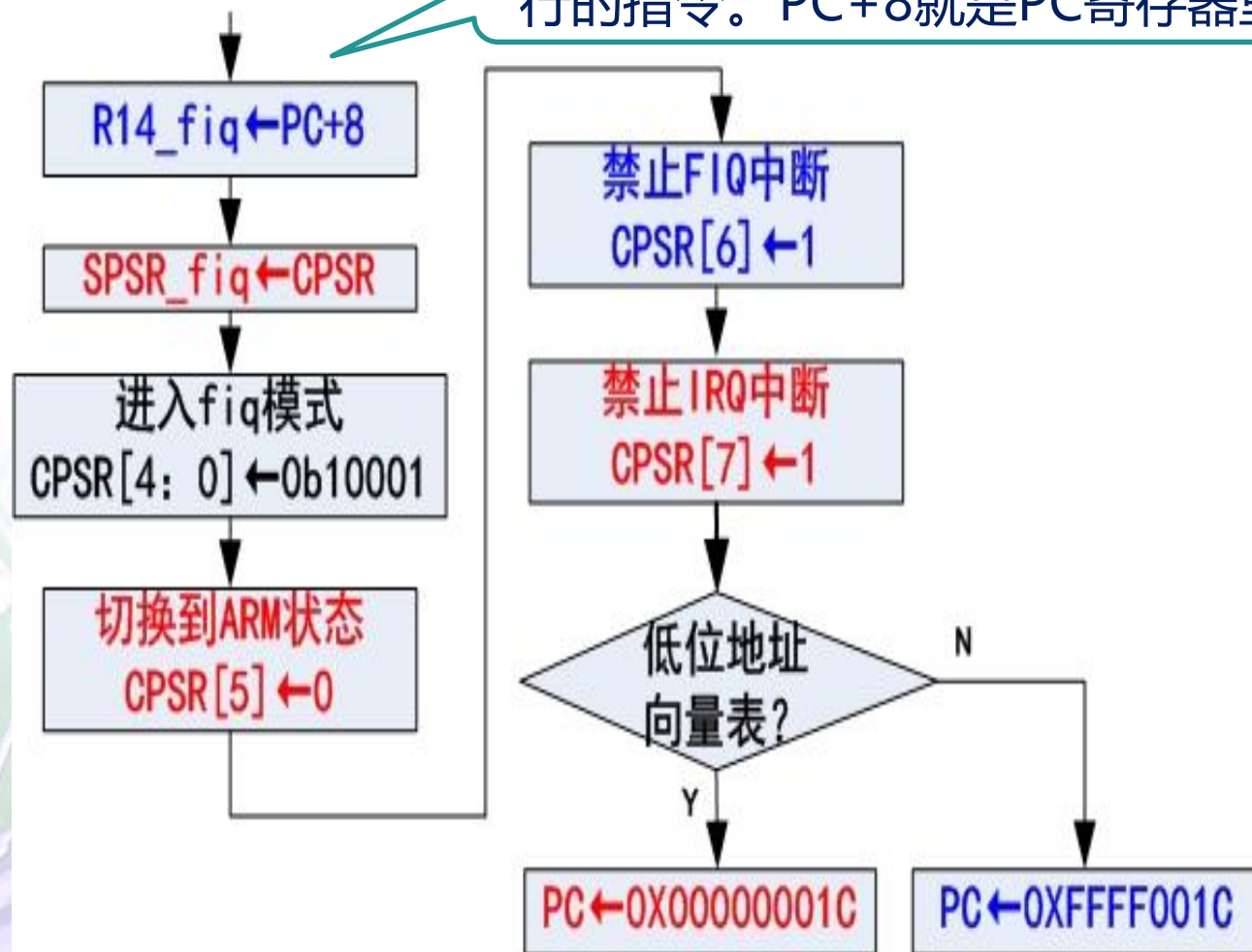
当F控制位被清零时，ARM在每条指令执行结束时检测FIQ同步器输出端的低电平。

FIQ异常进入与退出的流程与IRQ类似。

2.9 ARM的中断（异常）

FIQ的响应过程

这里的PC指向的是产生中断时，正在运行的指令。PC+8就是PC寄存器里的值



2.9 ARM的中断（异常）

四、未定义指令异常

未定义指令异常是**内部**异常中断。当ARM处理器遇到一条自己和系统内任何协处理器都**无法执行**的指令时，就会发生未定义指令异常，从而进入中断处理程序；

软件可使用这一机制，通过**仿真**未定义的协处理器来**扩展**ARM指令集。

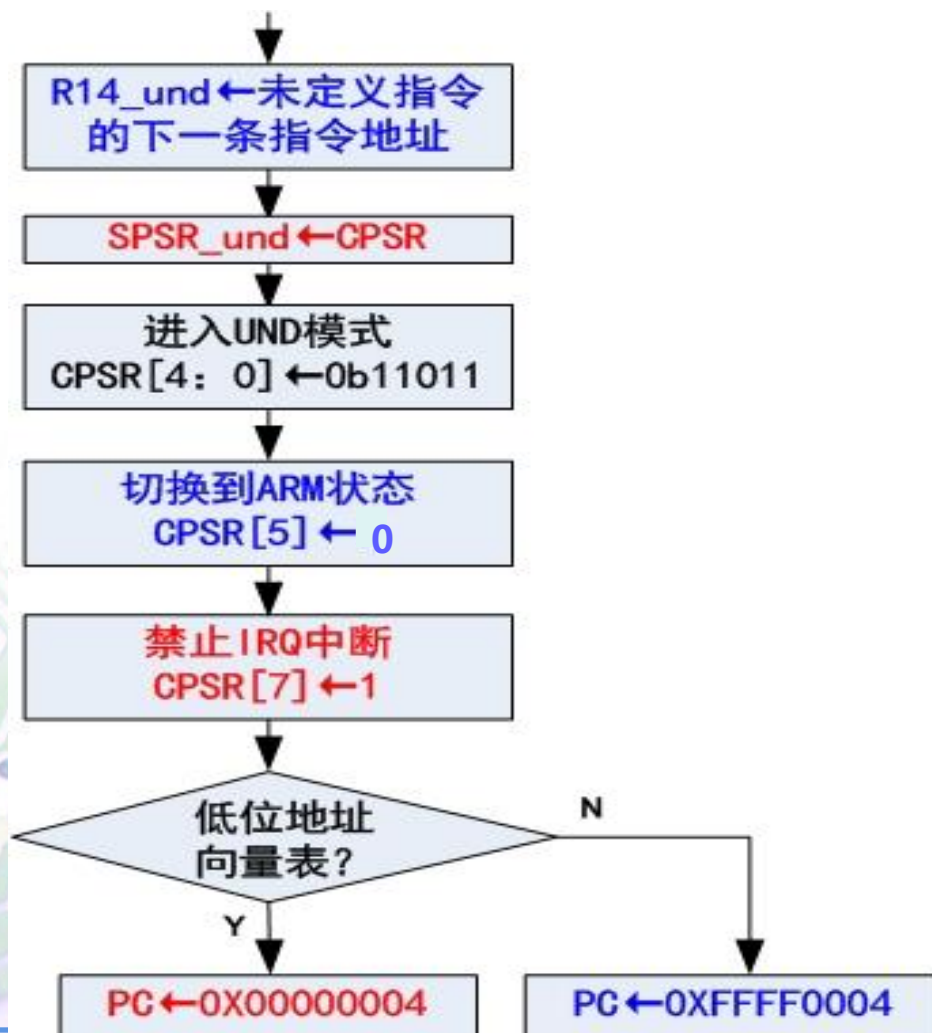
在仿真未定义的指令后，不管处于哪种处理器操作状态（ARM或Thumb状态），处理器执行下面的指令返回：

`MOVS PC, R14_und`

MOVS指令将R14的值写入PC，CPSR将自动从SPSR寄存器中恢复并返回到未定义指令之后的指令。

2.9 ARM的中断（异常）

未定义指令响应过程：



2.9 ARM的中断（异常）

五、中止异常

中止表示当前对存储器的访问不能被完成，这是由外部 **ABORT** 输入信号引起的异常中断。中止有2种类型：

- 预取指中止：由**程序存储器**出错引起的中止异常；
- 数据中止：由**数据存储器**出错引起的中止异常。

(1) 预取指中止异常：

当发生**预取中止**时，ARM内核将预取的指令**标记为无效**，但在指令到达**流水线的译码阶段**时才进入异常。如果指令在流水线中因为**分支**而**没有被执行**，中止将**不会发生**。

2.9 ARM的中断（异常）

在处理中止的原因之后，不管处于哪种处理器操作状态（ARM状态或者Thumb状态），处理程序可以通过执行下面的指令返回：

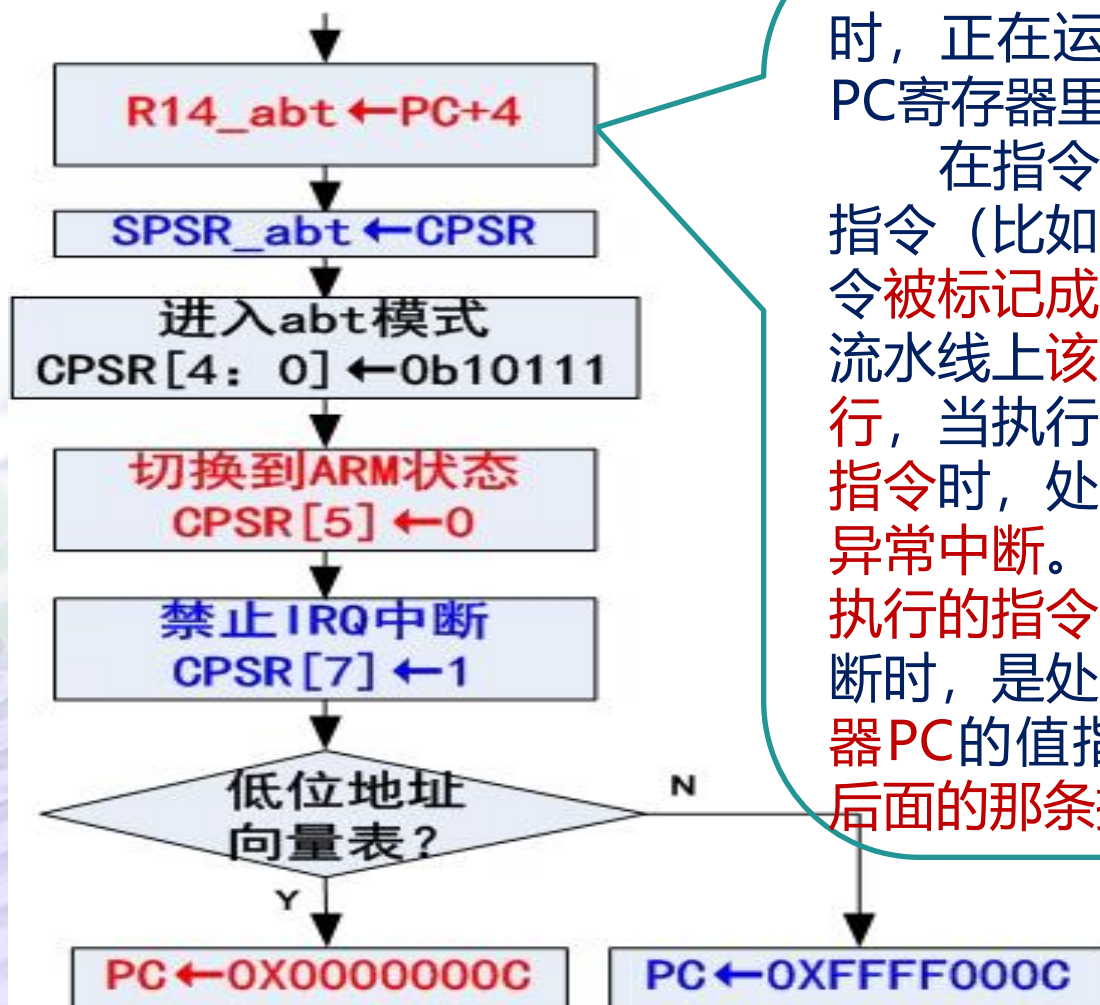
`SUBS PC, R14_abt, #4` ;在中止模式下，执行该指令返回

`SUBS`指令将R14的值减4写入PC，CPSR将自动从SPSR寄存器中恢复并**重试**被中止的指令。

BL	MOV PC,R14	PC+4	PC+2	此处PC为BL，SWI，未定义的指令取指或预取指中止指令的地址
软件中断 SWI	MOVS PC,R14_svc	PC+4	PC+2	
未定义的指令	MOVS PC,R14_und	PC+4	PC+2	
预取指中止	SUBS PC,R14_abt,#4	PC+4	PC+4	

2.9 ARM的中断（异常）

预取指令中止响应过程：



这里的PC指向的是产生中断时，正在运行的指令。PC+4就是PC寄存器里的值。

在指令预取时，无法正确读出指令（比如指令地址错误），该指令被标记成有问题的指令，这时，流水线上该指令之前的指令继续执行，当执行到该被标记成有问题的指令时，处理器产生指令预取中止异常中断。指令预取异常是由当前执行的指令自身产生的，当产生中断时，是处于译码阶段，程序计数器PC的值指向当前产生异常指令后面的那条指令。

2.9 ARM的中断（异常）

（2）数据中止异常：

数据中止异常表明数据存储器不能识别ARM处理器的数据读写请求，说明此次ARM处理器的读数据操作无效。

当发生数据中止异常时，理想的状况是进入数据中止异常的ISR，然后在内存中挑选出问题，再**重新执行**导致异常的指令。于是就会将PC返回2条指令，也就是将R14的值减去8，并将结果存入PC。

2.9 ARM的中断（异常）

在处理中止的原因之后，不管处于哪种处理器操作状态（ARM状态或者Thumb状态），处理程序可以通过执行下面的指令返回：

`SUBS PC, R14_abt, #8` ;在中止模式下，执行该指令返回

`SUBS`指令将R14的值减8写入PC（**返回并重新执行**导致异常的指令），CPSR将自动从SPSR寄存器中恢复并重试被中止的指令。

2.9 ARM的中断（异常）

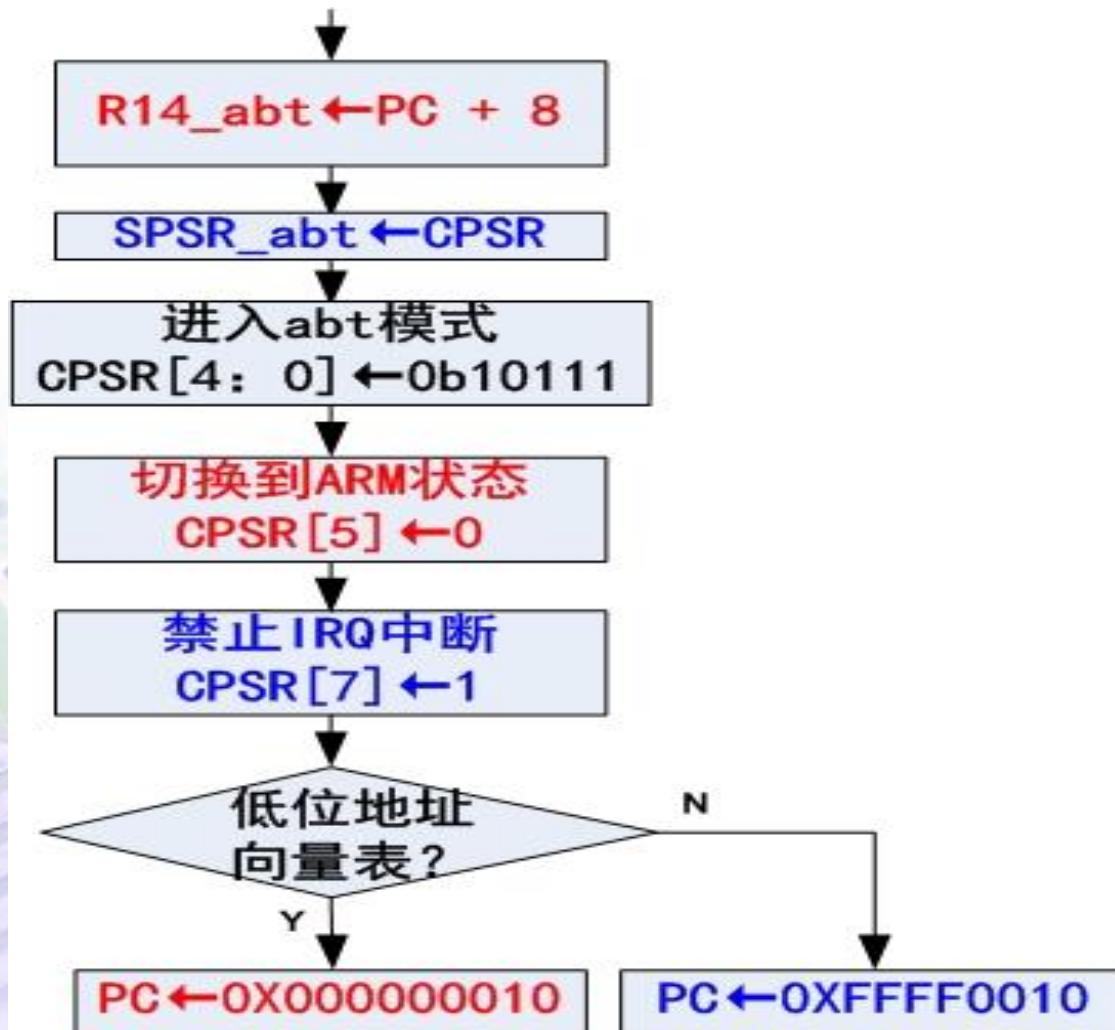
Abort（中止）主要用于**虚拟存储器**的管理中。在采用虚拟存储器的系统中，处理器可以产生**任意的**地址。当某个地址的**数据无效**，MMU（存储器管理单元）将产生一个**abort中止**。

思考:在**调试阶段**如果进入了**中止异常模式**，该如何找出问题所在呢？

可以通过中止模式下的R14找到进入异常前的程序位置。如果意外地发生了未定义异常，可以采取类似的处理方法。

2.9 ARM的中断（异常）

数据访问中止异常响应过程：



2.9 ARM的中断（异常）

六、SWI软件中断异常

事实上，所有的任务都是在用户模式下运行的，因此任务只能读CPSR而不能写CPSR。任务由用户模式切换到特权模式的唯一途径，就是使用一个SWI指令调用。SWI指令强迫处理器从用户模式切换到SVC管理模式，并且IRQ中断自动关闭，所以软件中断方式常被用于系统调用。只有处理器切换到系统模式时，IRQ中断才能继续使用。

由于它是由用户在程序中使用指令而产生的中断，所以叫做软中断。它也是所有中断（异常）中唯一的一个同步事件。

2.9 ARM的中断（异常）

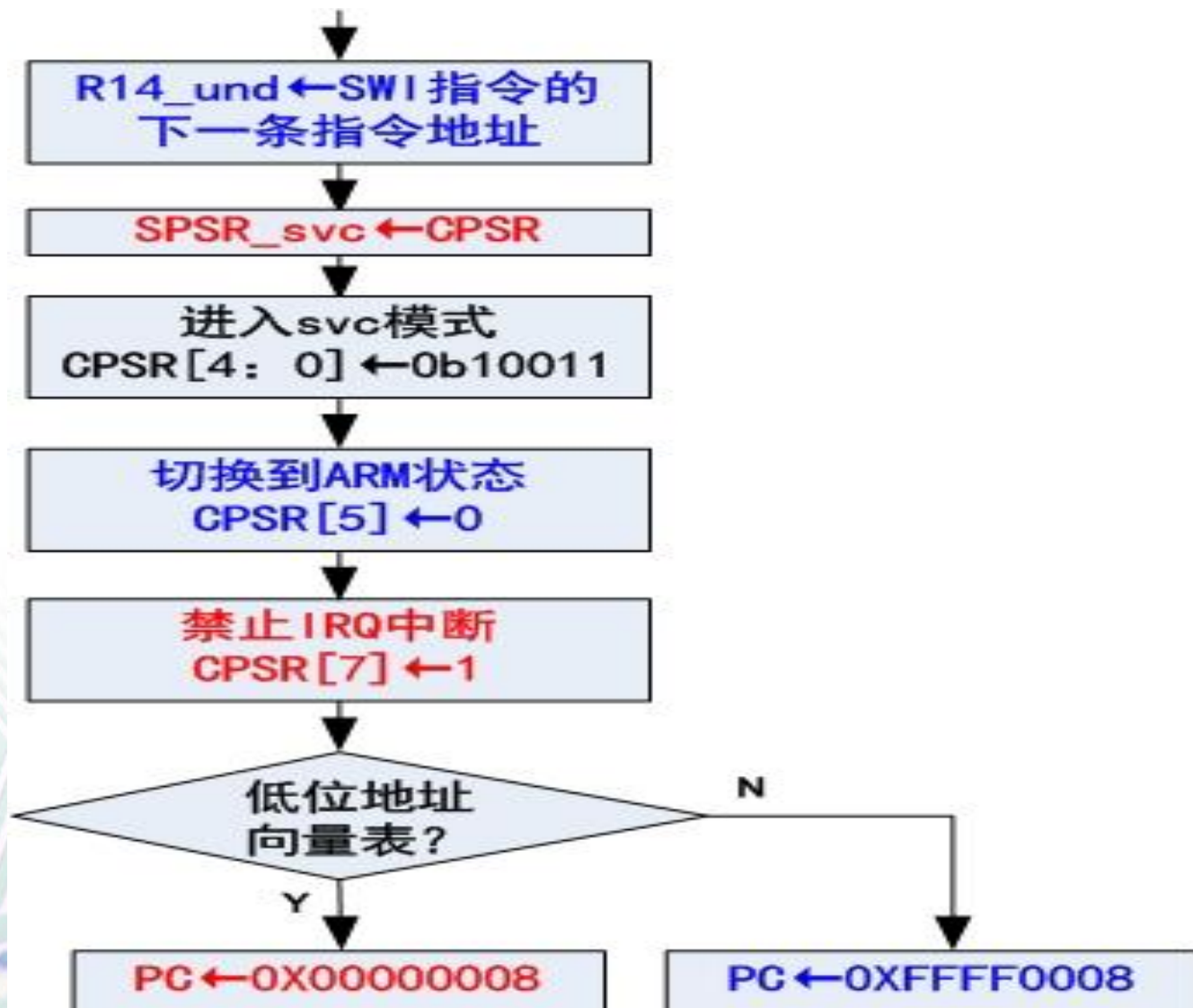
SWI处理程序通过执行下面的指令返回：

`MOVS PC, R14_svc` ;在管理模式下，执行该指令返回

MOVS指令将R14的值写入PC，CPSR将自动从SPSR寄存器中恢复并返回到SWI指令之后的指令。

2.9 ARM的中断（异常）

软中断响应过程：



作业

作业见群文件《平时作业要求》中第2章的内容。